Process of designing a new Hardware System for a client and its performance testing

Title: Hardware System Design and Performance Report for a Fictional Client

Word Count: 12827

Table of Contents:

1. **Introduction..... 3**
2. **Client Background and Requirements..... 3**
3. **Initial Investigation and Diagnosis..... 5**
4. **The Problem and The Solution..... 6**
    - 4.1 Processor(CPU)..... 6
    - 4.2 Memory(RAM)..... 14
    - 4.3 Storage..... 20
    - 4.4 Graphics(GPU) 22
    - 4.5 System integration (Motherboard, Bus, Cooling, OS)..... 23
    - 4.6 Auxiliary Devices..... 26
    - 4.7 Solution Conclusion..... 27
5. **Performance impact of the Solution..... 29**
6. **The Maintenance of the Solution..... 31**
7. **Alternative Architecture..... 32**
8. **Overview of the Testing Program/Application..... 34**
9. **Program Results Analysis..... 35**
    - 9.1 Testing Methodology..... 35
    - 9.2 Raw Testing Data..... 37
    - 9.3 Benchmarking results analysis and Limitations..... 39
10. **Conclusion..... 41**
11. **Reference..... 42**
12. **Appendix..... 49**
    - 12.1 A.1 – Clients Constraints and Current System..... 49
    - 12.2 A.2 – Matrix size Calculation and Interpretation..... 52
    - 12.3 A.3 – New system requirements..... 53
    - 12.4 C.1 – Source Code Listing..... 54

12.5 C.2 – Program Execution Output..... 66

1. Introduction

A fictional university research laboratory called Bellerman Molecular Research Centre is considering spending thousands of Euros on new memory and expects much faster calculation results – although there is no real evidence of the memory being the cause. This is the central problem that the university-affiliated research laboratory is facing, whose grant-funded simulation on molecules work has slowed down to bare minimum, while their IT department points towards insufficient RAM as the cause.

This is a report on the intervention as an independent hardware consultant asked to investigate this assumption. In this process the current hardware choices will be analysed and a diagnosis of the real issues of performance loss will be performed, examination and designing of the new hardware system for this research laboratory will be conducted and finally justification will be provided for the recommended replacements with research and a testing program being the main aim of this report.

This report is structured in two connected parts: Part A and Part B. Part A is the process of investigating the client's problem and justifying the correct solution. On other hand part B presents a C++ testing program with benchmarking which empirically tests and validates the proposed solution closing the gap between theory and practical implementation.

2. Client Background and Requirements

Bellerman Molecular Research Centre is a fictional university-affiliated research laboratory which specializes in molecular dynamics and more precisely the way that protein chains fold and interact together under different circumstances. Molecular dynamics is a branch of physics which extrapolates computational modelling and Newton's classical laws of physics into computationally intensive calculations proportional to the size of the system that is being modelled *(ScienceDirect Topics, n.d.)*. The university research itself supports a significant research grant, making results arrive on time a priority.

The core of the lab's research relies heavily on very large matrix multiplications. In molecular dynamics such simulations and analysis comes from the operations performed on the Hamiltonian matrix which is used to create the density matrix of the electronic structure of the system at each simulation step *(Djidjev et al., 2016)* allowing the researchers to understand the correlation between different protein chains in different scenarios (*Matlantis*, 2025). Since around 2006 the research centre has used a software that runs on the same PC since then. As time went by the matrices got larger and more complex. The Bellerman Molecular Research Centre pipeline is built around a proprietary molecular simulation suite, which performs matrix operations described above. They also plan to start keeping all past experiments on the computer and use different software for the purpose of analysis and comparison.

At the moment the university has a grant deadline and because of the old software they are incapable of handling such matrices. They finally decided to change the software to accommodate for new and faster ways of performing matrix calculations with the producer claiming it supports multithreading and more, referred to as ProteinSim 4. Unfortunately, during the first few simulations and modelling it was discovered that the speed of calculations did not improve at all which did not fix the issue of meeting the grant deadline.

The university operates under some significant constraints (*see Appendix A.1, Table 1*).

From the perspective of Bellerman Molecular Research Centre the problem is straightforward: as the new software was implemented, bigger matrices were used to create models and the hardware is insufficient. Their existing IT support team attributed this problem to insufficient primary memory and recommended upgrading RAM by increasing its amount and implementing higher speeds.

Finally, it leads to the objective of this investigation which is therefore to resolve the genuine problem with bottleneck of the university laboratory at minimal cost by correctly identifying which components need to be replaced and which can be reused.

3. Initial Investigation and Diagnosis

At the beginning in order to achieve the aim of this consultancy the problem had to be understood in-depth, and the true cause had to be found. The next step was to perform an investigation to diagnose the root of the problem with weak performance even after the software change rather than going with what their IT team suggested. First step was to analyse the current system and finally each component was in-depth understood (*see Appendix A.1, Table 2*) which forms the baseline for the diagnosis and later recommendation of the new system.

After an in-depth analysis of the current system, two findings that the current IT team did not consider, were identified as:

Finding 1: The software change did not help at all. This has its explanation in the CPU itself. The processor cannot parallelize or multithread because it has single core and a single thread with no multi-threading support. This shows that it is actually a hardware issue and not a software one. This explains why the software change did not bring the required and expected results.

Finding 2: The claim made by the IT team is not supported by data calculations and at the current matrix sizes the memory is not the problem. In order to test that claim the amount of required memory for one calculation was conducted. It is known that the university needs 3 matrices in one calculation. This is because a possible operation is a matrix multiplication (A x B = C). This means that matrix A and B needs to be loaded, and the result calculated, which means 3 matrices will be in the memory at once. One cell in a matrix is stored as a double-precision floating-point value, requiring 8 bytes of memory per element which is chosen by the research center as a conventional precision for CPU based calculations as it has a greater accuracy when compared to single precision data *(Stone et al., 2007)*. To calculate the number of bytes for one program the following formula is used *(Stone et al., 2007)*:

$$3 \times N^2 \times 8\ bytes$$

The equation was applied to matrix ranges given from the university laboratory and the results with interpretation and analysis was conducted *(see Appendix A.2, Table 3)*.

Concluding, the final diagnosis as to why the performance would not get better is firstly the fact that the CPU cannot run more than one instruction at a time making the hardware replacement unavoidable; secondly is the memory access pattern is inefficient during calculations which will continue to waste performance unless taken care of.

4. The Problem and The Solution

Now when the root cause of the problem is identified, it is clear that the hardware replacement is unavoidable because there is no software which is capable of doing parallel processing with only one execution unit. The proposed solution is divided by components and was sized against the requirements and so will be later used for referencing the components selections (*see Appendix A.3, Table 4*).

4.1 Processor(CPU)

The CPU is proposed to be changed from Intel Pentium 4 to a much more performant AMD Threadripper PRO 5965WX which has a total of 24 cores and is designed for exactly this kind of workload. In order to justify this choice, the comparison of the recommended and current CPU was conducted *(see Table 5)*.

Table 5

Comparison of the current and recommended CPU specifications

Specification	Current CPU *(Schmid, 2004)*	Recommended CPU *(AMD, n.d.)*
Architecture	x86(CISC), NetBurst	x86-64(CISC), Zen 3
Cores / Threads	1 / 1	24 / 48
Process Node	90nm	7nm
Base / Boost Clock Speed	2.80GHz (fixed)	3.8GHz / 4.5 GHz
L1 Cache	16KB for data	1.5MB total (32KB for data and 32KB for instructions per core)
L2 Cache	1MB unified	12MB total (512KB per core)

L3 Cache	None	128MB shared (32MB slices per chiplet)
Memory channels	2 (dual-channel)	8 (octa-channel)
TDP	89W	280W

From the table it can be noticed that all of the parameters differ between the old and new CPU, and each contributes directly to resolve the main issue.

The most significant difference in both CPUs is the core and thread count. This is because the current processor can only execute one instruction stream, while the recommended processor provides 24 physical cores and 48 threads. A core is an independent hardware component of a CPU which is isolated and can process set of instructions on its own, on other hand a thread represents a sequence of instructions that a single core can go through *(Walilko, 2025; Hennessy & Patterson, 2019)*. This means that a matrix calculation can be split across threads and run in parallel for the first time. This is the first step of removing the hardware ceiling which the software could not work around previously. What makes this improvement possible is the processor node which shrunk from 90nm to 7nm. This measurement describes the smallest component on the CPU and the smaller the node the more components can fit in one CPU *(Connatser, 2023)*. This shows how a fifteen-year transformation happened and why the processor can be so much more performant.

Clock speed in this two CPUs also differ but not a lot. Clock speed represents the speed so the cycles count in which each core can process instructions per second, measured in gigahertz *(Cloud, n.d.; Hennessy & Patterson, 2019)*. In this situation it changes from 2.80GHz to 3.80GHz. This will allow the processor to process instructions faster as the number of ticks in one second will be greater *(What Is Clock Speed & Why Does It Matter? | Lenovo NZ, n.d.)*. On the other hand, the greater part of the performance improvement comes from the core count rather than the clock speed. This shifts from increasing the clock speed towards increasing the amounts of core also reflects the industry trend. This was first noticed by Gordon Moore who observed that the number of components like transistors on a CPU integrated circuit doubles every approximately two years at a very minimal cost increase *(Moore, 1965)*. For many years, making transistors smaller allowed speeds to rise without increasing power density, which is known as

Dennard scaling. Around 2005 this stopped working because transistors started leaking current and the heat became too big to be cooled down. This made manufacturers shift and they started increasing the core counts, making parallelism possible rather than increasing clock speed without the proportional rise in electric power consumption *(Chen & Oyelami, 2024)*.

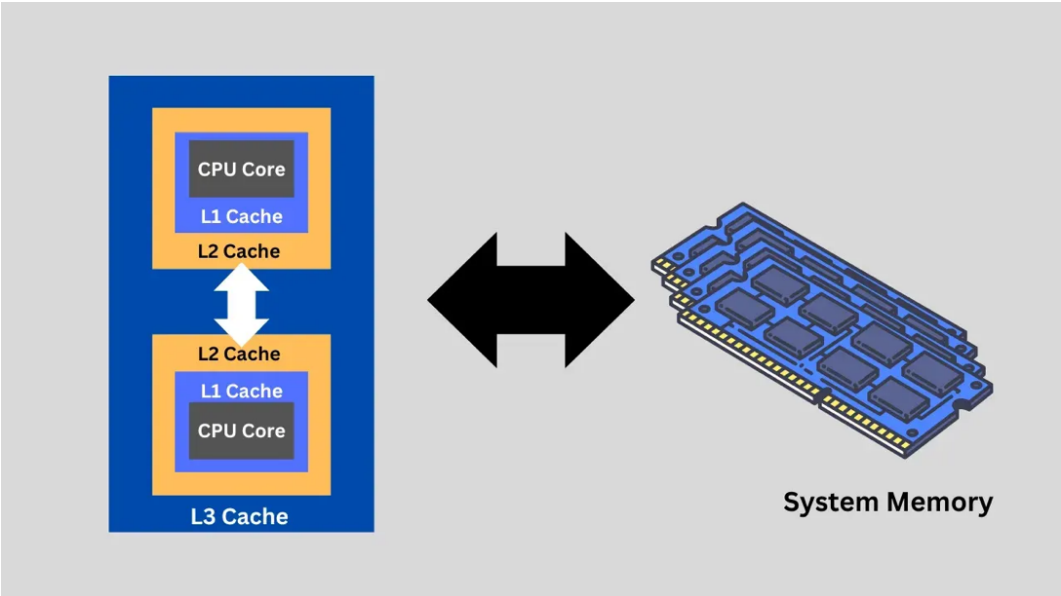
Regarding the architecture which refers to the instruction set that the CPU follows to carry out operations *(Hennessy & Patterson, 2019)*, both CPUs use CISC (Complex Instruction Set Computer) architecture specifically the x86/x86-64 which remains mostly the same. The only change is with the addition of 64-bit supports(x86-64). With the 32-bit x86 the system is limited to using only 4GB of memory with the architecture of the CPU, but this limit also comes from the chipset. The addition of 64-bit extension removes this memory ceiling with keeping the CPU architecture the same *(Hennessy & Patterson, 2019)*. CISC is also a type of architecture which favours execution of less but more complex instructions per operation. In comparison RISC (Reduced Instruction Set Computer) design such as ARM, favours more but less complex instructions at higher clock speeds *(Spiryn, n.d.)*. In this case the architecture was retained because the new software was built in x86 and migration to RISC would require whole software rewrite conflicting with a constraint set earlier *(see Appendix A.1, Table 1)*.

Now moving to cache and specifically about level one, two and three caches. Cache itself is a piece of memory which is fast and built directly into the processor which allows to store mostly accessed data and instructions reducing the need for the CPU to go to the main memory every time and so reducing the latency and wait time of CPU *(Lau, 2025; Hennessy & Patterson, 2019)*. The first level cache is private and separate for each core, the second level cache is a bit bigger but a bit slower and in the recommended CPU, it is also private (512KB per core)*(see Table 5)* and finally, the level 3 cache is the biggest and the slowest of all and it is shared across the cores of each chiplet *(see Figure 1 for a generic example)(Anderson, 2025)*. In the old CPU the caches were not enough for any matrix, on the other hand the new CPU has an L3 cache large enough to hold much of the working set allowing for very big improvement in performance recalling matrix size calculations *(see Appendix A.2, Table 3)*. In the case of primary memory, CPU has something called memory channels which is the amount of bus channels that CPU has

to communicate with RAM and the increase from two to eight allow for greater data transfer *(What Is a Memory Channel? RAM Performance Explained, n.d.)*.

Figure 1

Cache diagram in a CPU



Adapted from (Anderson, 2025)

Furthermore, the proposed new CPU uses a Zen 3 architecture which is optimized for deep pipelining and superscalar execution allowing multiple instructions to be executed in one clock cycle rather than one single instruction at a time. This with the increase in the core count allows substantially more work to be done per second. Moreover, the legacy system is based on the ICH6 chipset, which uses the legacy interrupt feature where multiple devices share a limited interrupt lines which created contention as the device count increases. The proposed modern chipset support MSI-X (Message Signalled Interrupts, Extended), which assigns to each device its own interrupt vector rather than sharing a small pool of lines *(Hennessy & Patterson, 2019)*. This significantly matters when more I/O devices are connected to the motherboard such as NVMe storage or networking devices.

Now it is important to estimate the expected performance improvement from the proposed processor. Firstly, the improvement from cache hierarchy can be estimated using the AMAT (Average Memory Access Time) measurement which is a metric that is used to compare different cache designs based on the time it takes to serve a request and the cost when it is not *(CS 61C Course Staff, n.d.)* and it is calculated as:

$$AMAT = L1\ Hit\ Time + L1\ Miss\ Rate \times$$

$$(L2\ Hit\ Time + L2\ Miss\ Rate \times$$

$$(L3\ Hit\ Time + L3\ Miss\ Rate \times Memory\ Access\ Time))$$

Where:

- Hit time is the amount of time it takes for data to be retrieved from the cache
- Miss Rate is the amount in percentages that data is not found in cache and needs to go deeper.

Now in order to be able to estimate AMAT for the old and new CPU, some numbers have to be assumed. The hit time for a modern processor for L1 cache is approximately 4 cycles, for L2 cache it is approximately 12 and for L3 it is approximately 44 cycles, and finally a typical primary memory access costing around 200 cycles *(Submillisecond, 2026)*. Secondly in order to estimate the miss rate of each cache, it was assumed that the working set is of 54MB calculated from the 1500x1500 matrix *(see Appendix A.2, Table 3)*. The estimated miss rate for each cache in the old and new system can be found below *(see Table 6)*. This method of calculation assumes that data is only found in cache if the whole working set fits inside of it meaning it ignores spatial locality. A cache line holds 64 bytes, or eight double-precision values, so reading values one after another would miss only about one in eight accesses (12.5%) even in a small cache. The miss rates in Table 6 are therefore worst-case estimates.

Table 6

Estimated worst-case miss rate for each cache level

CPU	Cache	Size	Hit Rate Estimation (cache size / 54MB	Miss Rate (100% - Hit Rate)
Old	L1	16KB	16/54,000 = 0.03%	99%
	L2	1MB	1,000/54,000 = 1.9%	98%
	L3	None	-	-
New	L1	32KB	32/54,000 = 0.06%	99%
	L2	512KB	512/54,000 = 0.93%	99%
	L3	32MB per chiplet (128MB total)	128,000/54,000 = 237% (exceeds working set)	10%(capped, rather than claiming a near zero miss rate)

Each core can only use the 32MB L3 slice of its own chiplet. Every thread needs the whole of matrix B (18MB at 1500×1500) plus its own rows of A and C, which fits within one 32MB slice, so a 10% miss rate is retained.

The current AMAT can be calculated as:

$$AMAT_{Curr} = 4 + 0.99 \times \left(12 + 0.98 \times (200)\right)$$

$$AMAT = 209.92\ cycles$$

The recommended AMAT can be calculated as:

$$AMAT_{Rec} = 4 + 0.99 \times \left(12 + 0.99 \times (44 + 0.1 \times 200)\right)$$

$$AMAT = 78.61\ cycles$$

Finally, the AMAT reduction factor can be calculated by:

$$AMAT_{Reduction\ Factor} = \frac{AMAT_{Curr}}{AMAT_{Rec}}$$

$$AMAT_{Red\ Fac} = \frac{209.92\ cycles}{78.61\ cycles}$$

$$AMAT_{Red\ Fac} = 2.67$$

From the calculation above it can be noticed that the average memory access time is 2.7 times lower, a reduction of 62.6% only from the cache change itself with no core or multi-threading addition. It is also very important to mention that these are the worst-case figures derived from the size of the working set which can be smaller or bigger depending on the given experiment. The actual empirical data can be found in Section 9.

Finally, the expected performance gain from parallelism was calculated using the Amdahl's Law which estimates the speedup achievable by parallelising some portion of the program/software used for calculating the matrices *(Amdahl, 1967)*. The formula is as follows:

$$Speedup = \frac{1}{((1 - P) + \frac{P}{N})}$$

Where:

- P is the proportion of the software/program that can be parallelised
- N is the number of cores available

The proportion of the program that can be parallelized needs to be derived directly from what the program does. When doing matrix operations like multiplication it is classified as an embarrassingly parallel problem where each thread can work independently. This is because a matrix can be split into smaller pieces and calculated once in multiple threads and cores. The only part that cannot be parallelized is the process of creating the matrix like inserting the data, the process of putting together the matrix after calculation and the final calculated output. The exact value of "P" was assumed to be 0.95 as an illustrative value, because the serial part of the program is very small compared to the multiplication itself. The actual value of "P" will be calculated in section 9.

Before calculating the theoretical speedup, a design target was set. As said before(*see Appendix A.3, Table 4*), the processor must provide a speedup of at least 10 times. The minimum required number of cores to reach this speed can be calculated with Amdahl's Law as:

$$\frac{1}{(0.05 + (0.95 \div N))} \geq 10$$

$$0.95 \div N \leq 0.05$$

$$N \geq 19$$

As it can be seen, at least 19 cores is needed however, 19 meet the requirement only if P is 0.95 and so there is no margin, but processors come in fixed core counts. Therefore the 24-core processor was chosen. Also, the N is not the number of threads but cores, and because one core can share two threads, so 48 threads is treated as extra headroom. With the 24 cores the theoretical speed can be calculated by:

$$Speedup = \frac{1}{((1 - 0.95) + \frac{0.95}{24})}$$

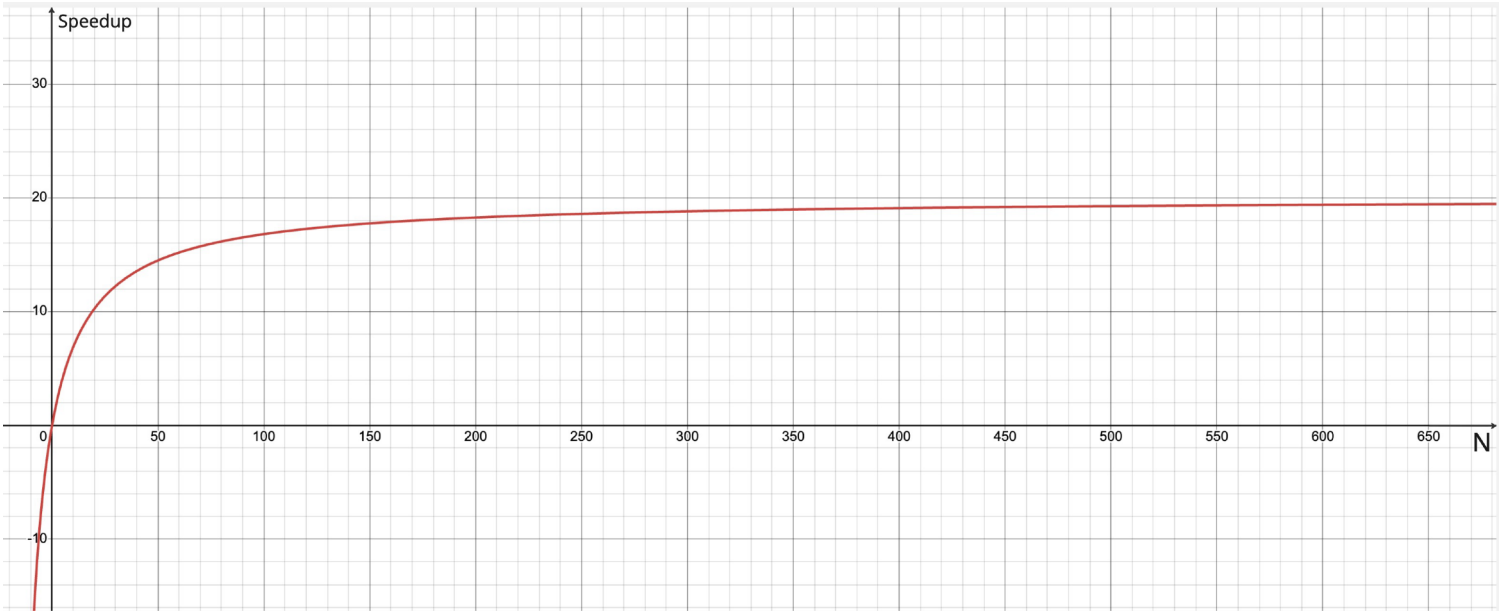
$$Speedup = 11.2x$$

As it can be seen the speedup comes up to only 11.2 times and not 24 as it would be assumed with the 23 additional cores to the one which was there before. This is normal because of the 5% serial portion. Amdahl's law shows that even with infinite number of cores, the 5% serial portion, limits the maximum speedup to 1/0.05 so 20 times the value compared to the old CPU with single core. This can be seen on *Figure 2* when the following equation is plotted:

$$Speedup = \frac{1}{((1 - 0.95) + \frac{0.95}{N})}$$

Figure 2

Speedup equation on a graph



Created by the author with Miro and Desmos Calculator

Before the final recommendation is made, the CPU was also compared with an alternative option *(see Table 7)*.

Table 7

Comparison of the recommended and alternative CPU specifications

Specification	Recommended CPU *(AMD, n.d.)*	Alternative Intel Xeon W7-3455 *(CPU-Monkey, 2025)*	Impact on the Client
Core / Threads	24 /48	24 /48	No impact
Base / Boost Clock	3.8GHz / 4.5GHz	2.5GHz / 4.8GHz	Higher sustained clock on the recommendation

L3 Cache	128MB shared (32MB slices per chiplet)	67.5MB shared	The recommendation has about twice the total L3 cache directly affecting AMAT
Memory Channels	8 (DDR4 ECC)	8 (DDR5-4800 ECC)	Equal channel count but alternative supports newer memory standard
TDP	280W	270W	Very comparable
Memory bandwidth (8 channels)	204.8GB/s (DDR4-3200)	307.2GB/s (DDR5-4800)	The alternative has 50% more bandwidth
Vector instructions	AVX2	AVX-512	The alternative can process more data per instruction
CPU Price only	€2,135 (Tom's Hardware, 2022)	€2,215 (CPU-Monkey, 2025)	The recommendation is cheaper

Both CPUs offer very similar specifications like the core and threads counts, and TDP. The alternative has two main advantages, the first one is its eight channels of DDR5-4800 memory which gives a theoretical bandwidth of 307.2GB/s, which is around 50% more than the recommendation, and it supports AVX-512 instructions, which can speed up floating-point calculations. However, Section 3 identified the cache hierarchy as one of the major issues, and the recommendation provides about twice the L3 cache (128MB against 67.5MB), a higher base clock and a lower CPU price. Therefore, the recommendation is retained.

4.2 Memory(RAM)

Now having established the CPU was the primary constraint and having the new CPU recommended, the attention can be turned into memory, the component that was firstly blamed to be the real problem by the client's IT team. The recommendation is to change the old 4GB DDR1 SDRAM with 64GB of DDR4-3200 ECC Registered memory which will populate all eight memory channels allowed by the new CPU.

Table 8

Comparison of the current and recommended memory specifications

Specification	Current (Kingston ValueRAM KVR400X64C3A/1GB) *(Icecat, n.d.)*	Recommended (Kingston Server Premier KSM32RS8/8MR) *(Kingston, n.d.)*

Amount of RAM sticks	4	8
Capacity / total	1GB / 4GB	8GB / 64GB
Type	DDR1 SDRAM (non-ECC)	DDR4 SDRAM, ECC Registered
Speed	400MT/s (PC3200)	3200MT/s (PC4-25600)
Latency	CL3	CL22
Channels Used	2(dual-channel)	8(octa-channel)
Error Correction	None	ECC (single-bit correct, multi-bit detect)

It can be observed that the recommendation has a bigger total capacity even though it was pointed out that the capacity is not a problem, and this is done deliberately not for capacity reasons. As shown in Section 3, the biggest matrices used today, a 2000x2000 uses at most 96MB, so under 2.5% of the current setup. The expected 10,000x10,000 matrices need at least 2.4GB so only 60% (see Appendix A.2, Table 3) and so showing that capacity is not the reason for this recommendation, but it will help with future.

The increase in the memory capacity is justified entirely with a different requirement. This is the fact that the CPU supports up to 8 independent memory channels. It is very important to use all eight channels because bandwidth scales with the number of channels used. As calculated in earlier section, the 11.2x parallel speedup increase can only be approached when all channels are used. This is because each channel acts as a 64-bit data path and the CPU can only work in 100% efficiency when a steady supply of data to each core is available *(Memory Systems Authority, n.d.)*. If the CPU does not receive a steady stream of data it will wait for data (called a stall), on top of that adding more cores will cause more narrow access to each path making this the best combination *(McKee, 2011)*. In order for the CPU to use all eight it needs to be populated so this number of modules needs to be used *(AMD, n.d.)*. One of the smallest commonly available ECC Registered DDR4 modules is 8GB, so eight modules give 64GB so a total populated capacity of 64GB and so meaning that the size increases as a side effect.

Speed of RAM is usually expressed in MHz (megahertz) or more formally as MT/s (megatransfers per second) which describes the data transfer rate of a single memory module *(Corsair, n.d.)*. It increases from 400 to 3200 which is an 8x improvement.

CAS Latency is the time so the amount of clock cycles between a data being requested and the start of data being returned. In this case the old and recommended RAM latency increases numerically from CL3 to CL22. This appears to be a straightforward improvement but in reality the true latency is calculated using also the clock speed and not only the CAS value *(Kingston, 2025)*. This is done as follows:

$$True\ Latency = CAS\ Latency\ \times \frac{2000}{Memory\ speed\ in\ MT/s}$$

The current true latency can be calculated as:

$$TL_{Curr} = 3\ \times \frac{2000}{400}$$

$$TL_{Curr} = 15ns$$

The recommended true latency can be calculated as:

$$TL_{Rec} = 22\ \times \frac{2000}{3200}$$

$$TL_{Rec} = 13.75ns$$

As it can be noticed, the true latency is marginally faster in the recommended system. This is because the CL is way bigger, but the clock speed is also bigger. This shows how CAS latency alone is misleading, and the true latency is different.

Now moving to the memory channels which are independent data pathways between the processor and RAM. If there are more channels, more data can be sent at the same time from and to RAM. In this case there is an increase from two to eight and each of them is 64-bit wide which means that this increase, increases the width of data path and not only the speed at which data is being sent *(Memory Systems Authority, n.d.)*. If not for the number of channels and let's say there would be only two, the 24-core CPU would quickly starve from data lack and would receive only 25% of the bandwidth.

Moving to one of the last specifications of the memory, being the ECC (Error-Correcting Code). ECC is an error correction mechanism built into the memory which was lacking in the old component. ECC works by using additional parity bits to detect and automatically correct single-bit errors before they affect a running calculation *(Crucial,*

n.d.). This is done through continuous scan of the stored data in the memory *(Crucial, n.d.)*. In the case of the Bellerman Molecular Research Centre where they do complex matrix calculations, having an error detection mechanism is very important duo to possible results inconsistencies affecting the whole analysis process. These errors may happen silently, and no analyst would be able to detect them on their own.

Now moving on to the next part which is the bandwidth of the new memory. Theoretical bandwidth is the maximum amount of data (rate) that can be sent and received through a bus between CPU and memory *(Memory Systems Authority, n.d.)* and can be calculated as follows:

$$Bandwidth = Transfer\ Rate\ \left(\frac{MT}{s}\right) \times 8\ bytes\ \times Nr\ of\ channels$$

Where:

- Each channel is 64 bits (8 byte) wide *(Memory Systems Authority, n.d.)*
- Transfer rate is the number of transfers per second

The current bandwidth with DDR1-400 with two channels can be calculated as:

$$B_{Curr} = 400 \times 8\ bytes \times 2$$

$$B_{Curr} = 6{,}400MB/s\ = 6.4GB/s$$

The recommended bandwidth with DDR4-3200 with eight channels can be calculated as:

$$B_{Recom} = 3200 \times 8\ bytes \times 8$$

$$B_{Recom} = 204{,}800MB/s\ = 204.8GB/s$$

It can be noticed that this presents an around 32x improvement increase which correlates with the increase in the CPU core count. If not for this, many cores would have to share a single path to the memory causing the width of the path to actually shrink per core. All of these are peak theoretical figures and most likely all will be lower.

On top of that, the developer suggested that the application in order to multiply w matrices, it uses three loops. The amount of data moved can be calculated with the following equation*(Demmel, 2016)*:

$$Total\ moved\ size = (n^3 + 3n^2) \times 8\ bytes$$

$$Total\ moved\ size = (10000^3 + 310000^2) \times 8\ bytes$$

$$Total\ moved\ size = 8 \times 10^{12}$$

$$Total\ moved\ size = 8TB$$

This calculations shows that the total moved data size will be around 8TB. This is a lot of data. With the recommended bandwidth at peak, the time it takes can be calculated by:

$$Time = \frac{size}{speed}$$

$$Time = \frac{8000GB}{204.8GB/s}$$

$$Time = 39s$$

From the calculation it can be noticed that in the future it may take up to 39s for only moving the data around. This means that at the moment there is no problem but in the future it may appear if insufficient RAM will be bough. If for example there would be only 4 channels used the time would be 1.3min and with only 2 channels up to 2.6min. These are peak bandwidth numbers, and the real data will be lower for sure meaning that the 8-channel recommendation is correct and save to do.

Virtual memory is yet another technique correlated with primary memory. Virtual memory is an OS functionality which uses part of secondary storage as primary memory in a case where there is insufficient memory *(Tutorialspoint, n.d.)*. In more depth actually what happens is a system-level trick in which the OS creates a fake, private memory map so it thinks, it owns a massive primary memory block *(Tutorialspoint, n.d.)*. When there is heavy dependence on virtual memory there is a downgrade of performance, because of way slower speeds of secondary storage. Fortunately for this client, there is more than enough RAM to store the calculation matrices in cache.

The last mechanism used by the memory is the Direct Memory Access (DMA) which allows peripheral devices such as storage and network cards to transfer data directly to and from memory without the need to engage CPU which would have to copy

and send data blocks to RAM *(GeeksforGeeks, n.d.)*. In the proposed new system, DMA is also used to retrieve matrices from NVMe storage and network but without interrupting the CPU's 24 cores meaning that at the same time an operation may be happening and another one can be starting although these transfers share the same memory bandwidth as the cores.

Lastly in order to confirm that the recommended RAM configuration is adequate an analysis was conducted *(see Table 9)*.

Table 9

Comparison of the recommended and alternative memory option

Specification	Option A: 8 x 8GB	Option B: 4 x 8GB
Total capacity	64GB	32GB
Channels populated	8 of 8	4 of 8
Theoretical bandwidth	204.8GB/s	102.4GB/s
Free DIMM slots	0	4
Cost	€1,276	€638

Option B has some advantages. The first is its price, which is half that of Option A (€638 compared with €1,276)*(TECH-R-US Store, 2026)*. The second is that 32GB is still far beyond what is needed at the current moment. Lastly, it leaves 4 DIMM slots free for future expansion. On the other hand, it has some disadvantages which are related to bandwidth. With only four of the eight channels populated bandwidth drops by half to 102.4GB/s. With these any DMA connected devices and the CPU share a halved path width. In this case, the CPU cores will be starved and the calculated speedup of 11.2x will not be reached. This concludes that, option B is not totally inadequate for the matrix sizes tested today. If in near future as told by the client in Section 2, they will expand and the matrices will get larger, at some point they will not fit in cache entirely. In that case it is very important to be able to retrieve data efficiently from RAM in order not to cause any latency. On top of that the laboratory may be doing few calculations at once and that will for sure cause problems as it all would not fit in the cache at all. This is why Option A is still the better choice.

4.3 Storage

Now, when the processor and memory issues were addressed, the next component which will be analysed is storage. As it was discussed earlier the storage was never a problem in the performance loss. It is possible to leave the old storage device, but it is not recommended. As the matrices will grow, more calculations will be performed, more data will be stored. Moreover, the laboratory needs protection of the data in case of some failure. The recommendation is to replace the current 80GB HDD SATA disk with two 1TB NVMe SSD drives mirrored in a RAID 1 set up. The current and recommended storage were compared *(see Table 10)*.

Table 10

Comparison of the current and recommended storage specifications

Specification	Current (Seagate Barracuda 7200.7 ST380817AS)*(Hard Disk Sentinel, n.d.)*	Recommended (2x Samsung 990 PRO 1TB)*(Samsung Electronics, 2022)*
Type	HDD, 7200RPM	NVMe SSD, flash memory
Interface	Serial ATA/150	PCIe 4.0 x4, NVMe 2.0
Capacity	80GB	1TB per drive (1TB usable)
Max Transfer Rate	150MB/s	Read: 7,450MB/s Write: 6,900MB/s
Access Time	12.67ms (8.5ms seek + 4.17ms rotational latency)	0.045ms (22,000 IOPS at queue depth 1)
Redundancy	None	RAID 1 (mirrored drive)
RAID Setting	None	1TB usable and 1TB for mirroring

As it can be noticed, the new recommended storage device is different in every parameter. The first major difference is the type of storage. The current is an HDD (Hard Disk Drive) which saves data on spinning platters with the use of a read/write head. On the other hand, the recommended drive is an SSD (Solid-State Drive) which stores data in a flash memory with no moving mechanical parts which makes it less exposed to mechanical failure and lower latency (Western Digital, n.d.).

The next difference is in the interface. The HDD drive uses a serial ATA/150 which limits the drive to only 150MB/s *(Hard Disk Sentinel, n.d.)* although a real hard disk delivers less than this in practice. The recommended NVMe disk communicates directly

with CPU through PCIe lanes and in this case it uses four of them which can reach up to 7,450MB/s of read and 6,900MB/s of write speeds *(Samsung Electronics, 2022)* which is roughly 50 times more. The time needed for access shows this difference more clearly. The current drive needs on average of 8.5ms to seek and 4.17ms of rotational latency, giving 12.67ms before any data moves *(Hard Disk Sentinel, n.d.)*. The recommended drive can serve up to 22,000 random reads per second at queue depth 1 so 0.045ms(1/22,000s) of average time per read *(Samsung Electronics, n.d.)*. This is around 280 times shorter.

The capacity of the drive also increase from 80GB to 1TB. Although this was not an issue the university plan is to keep all the experiments and results meaning as time goes by and their experiments expand there will be enough space.

The storage effect on performance is minor, but it has to be precisely outlined. In this case it affects the speed in which matrices are stored and retrieved. When taking the 1500x1500 matrix, for the current drive it is:

$$54\text{MB} \div 150\text{MB/s} = 0.36\text{s}$$

And for the recommended:

$$54\text{MB} \div 7{,}450\text{MB/s} = 0.0072\text{s}$$

This means that there is a difference, but as suspected this is not the bottleneck of the system since both are short.

Both drives are configured in a RAID 1 set up which will write data at the same time to both disks. In a case of a failure, all data is stored on both drives and capable of being retrieved if needed *(Patterson et al., 1988; IONOS, 2023)*. RAID 1 was selected because it writes data to both as it would write to one meaning no loss in performance will be observed. Although RAID 1 can protect from disk failure it cannot protect from accidental deletion, corruption or malware because these are copied on both disks immediately. Therefore, this is not a backup solution.

In order to confirm the recommendation a cheaper version was analysed without the use of RAID 1, *(see Table 11)*.

Table 11

Comparison of the recommended and alternative storage option

Specification	Option A: 2x 1TB SSD with RAID	Option B: 1TB SSD no RAID
Usable Capacity	1TB	1TB
Fault Tolerance	Survives one failure	None
Cost *(Samsung Electronics, n.d.)*	€570	€285

As it can be observed Option B has the advantage of half the price. The disadvantage is that it has no fault tolerance which means that in case of a failure data created since the last backup would be lost. Option B is also wors because in case of failure the whole OS will not be available until the disk will be replaced. This also means that the real cost of Option B is not going to be only €285 but also the risk to the grant deadline. This means that the recommendation stays as it was with the first option.

4.4 Graphics(GPU)

Now moving to the recommendation of the graphics(GPU) for this system. The recommended CPU has not built in GPU meaning that the system needs one in order to display any image, application etc. on the monitor. The current system uses a built in GPU with shared memory from RAM whereas the recommendation has its own memory, it's a professional and low-cost GPU, sufficient for displaying purposes (see Table 12).

Table 12

Comparison of the current and recommended GPU specifications

Specification	Current (Intel GMA 900) *(Hinum, n.d.)*	Recommended (NVIDIA RTX A400) *(NVIDIA, n.d.)*
Type	Integrated in chipset	Discrete PCIe card
Video Memory (VRAM)	Not available (shares RAM)	4GB GDDR6
Memory bandwidth	Shares RAM	96GB/s
Processing units	4 pixel / 0 vertex shaders	768 CUDA cores
Max power	Part of chipset	50W
Price	With motherboard	€205

The benefit of the new GPU needs to be stated precisely because it does not take part in the calculations process although there is an effect. Because the GPU has its own

VRAM, it does not eat up as before the shared RAM meaning more memory is available for use by the calculation software and whether the GPU could accelerate the calculation in Section 7. It is important to mention that because in other case, if there would be no VRAM the calculations made in the memory section would not be true as some part would be constantly used by the GPU.

4.5 System integration (Motherboard, Bus, Cooling, OS)

All of the components specified so far needs to work together and create a sophisticated complete system. This part will cover the motherboard that connects all components, busses that carry data and instructions, the Power Supply Unit (PSU), cooling system and finally the Operating System (OS).

Firstly, the motherboard recommendation was conducted through an analysis of the needs and other components that need to work together *(see Table 13)*.

Table 13

Comparison of the current and recommended motherboard specifications

Specification	Current (Intel D915GAG) *(The Retro Web, n.d.)*	Recommended (ASUS Pro WS WRX80E-SAGE SE WIFI) *(ASUS, n.d.)*
Socket / Chipset	LGA775 / Intel 915G	sWRX8 / AMD WRX80
Memory slots	4x DDR1, up to 4GB support	8x DDR4, 8-channel, up to 2,048GB support
Expansion slots	- 2x PCI - 1x PCIe x1 - 1x PCIe x16	- 7x PCIe 4.0 x16
Storage inputs	- 4x SATA	- 8x SATA III - 3x M.2 PCIe 4.0 x4 - 2x U.2
Network	- 1x LAN	- 2x 10G Ethernet
Cost	(Not applicable today because no longer in production)	€890

From the table above it can be noticed that every parameter in the current and the recommended motherboard are different. Firstly, because the socket and chipset are completely different in the CPU, the motherboard, cooling and memory must all be replaced together. The new motherboard has also support for eight channels DDR4 ECC

registered memory so twice the number of the current one *(ASUS, n.d.)*. The motherboard finally supports the NVMe storage which is faster and also support bootable NVMe RAID array *(ASUS, n.d.)* so the option on RAID 1 from Section 4.3 is supported. Moving on the 10G Ethernet ports allow for high-speed network connection and the BMC allows the computer to be monitored and restarted remotely. There is only one disadvantage of this solution being the size. The motherboard is built to be mounted only in a full-size rackmount case or a tower *(Puget Systems, n.d.)* and so the current case cannot be used for this new set up and needs to be replaced.

The motherboard uses also a specialized types of buses called PCI which is a shared parallel bus which is shared across all devices for data transfer. In this case the bottleneck may occur when too many devices want to use the same path with too many data. On the other hand, there is also a PCI Express which is a type of a point-to-point path between only two devices and no other device can share the same path. The recommended motherboard uses the express version allowing for these bottlenecks to disappear. The PCIe is also faster on the new motherboard when compared with the current. The motherboard also has a lot of different ports. Some of them will be taken by the recommended parts but there is still a lot of room for growth. There are possibilities of expanding the storage capacity, putting more graphics or a much more performant one and the RAM can be changed to provide a sum of up to 2TB of memory.

In the case where multiple devices want to use the same bus to send and receive data, bus arbitration comes into play. Bus arbitration is a centralized component which receives all requests and grants access to the bus for the purpose of transmitting data. This is a simple and low-cost method but also a single point of failure and a possible bottleneck *(HMS Networks, 2024)*. PCI uses a centralized system in which each bus master has its own grant and access requests to the bus arbiter. PCIe on other hand uses arbitration per link therefore centralized arbitration is a better option for the recommended motherboard. This is because each device has its own link meaning that the weakness of centralized arbitration matter little but low cost and simplicity remains.

Now moving away from the motherboard specification into the Network Controller that sits inside the motherboard. The controller has dual 10G Ethernet ports provided by the Intel X550-AT2 controller *(ASUS, n.d.)*. The current set up was a single Gigabit port.

The recommended one can carry up to (10 / 8 = 1.25GB/s) which is ten times the 0.125GB/s from the old setup. This can help with sending large data matrices across the facility network in the university.

The next step is to discuss cooling. The current set up has a very low need for cooling because of single core CPU and TDP of 89W which is provided by the basic intel boxed fan. The recommended CPU has a TDP of 280W *(AMD, n.d.)* with 24 cores, so it needs proper cooling. The recommendation here is to buy Noctua NH-U14S TR4-SP3 which supports the sWRX8 socket and has six heat pipes with a 140mm fan (Noctua, n.d.). In this setting it is not rated for 280W and that is why a second fan is added of NF-A15 which will allow to use this heatsink cooling for this CPU. The total cost for that would come to €144 (PCPartPicker, n.d.).

Now moving from the cooling and going into the Power Supply Unit (PSU) which in the case of recommended set up will draw far more power than the current one *(see Table 14)*.

Table 14

Estimated Power consumption of the recommended setup

Component	Model	Power
CPU	AMD Threadripper PRO 5965WX	280W
GPU	NVIDIA RTX A400	50W
RAM	8x Kingston Server Premier 8GB DDR4-3200 ECC	13.4W
Disk	2x Samsung 990 PRO 1TB	11.0W
Network Controller	Intel X550-AT2 (dual 10GbE, on the board)	11.0W
Cooling	Cooler fan (1.19W) + second fan (0.96W)	2.2W
Motherboard and other items	Motherboard circuitry, BMC, case fans, peripherals	60W
Total		**428W**

Requirement of the 60W in the motherboard and other items is an assumption as there are not specified power usage by only the motherboard itself. This is to make sure that there is enough power in the supply unit for any expansion

The recommended supply is a Corsair RM850x with 850W output and Cybenetics Gold efficiency at the price of €125 *(Corsair, n.d.)*. The recommended PSU will run at around half of its capability which is a greater room for expansion and changes. The current PSU would be used in around 99% which is not suitable and should not be conducted for reasons of peaks which wouldn't be supported by the PSU.

Moving to one of the last parts which is the case. Because the motherboard is an SSI-EEB board it means it is larger and too big for standard cases. This is why the recommended case is the Phanteks Enthoo Pro 2 Server Edition, which supports SSI-EEB motherboards and offers 11 expansion slots and up to 15 fan positions *(Phanteks, n.d.)*. The case accepts CPU coolers up to 195mm and GPUs up to 503mm long which is suitable for the recommendation. The case also supports a left side bracket designed to allow better air flow between GPU and CPU. The case costs around €160.

The last part that needs to be considered is the Operating System. Unfortunately, at the moment the OS is Windows XP released in 2001. The recommended CPU will not work with this OS because, AMD itself says that the CPU supports only 64-bit Windows 10 and 11*(AMD, n.d.)*. On top of that the way that interrupts works is guided by the OS and because Windows support of message-signalled interrupts (MSI and MSI-X) starts with Windows Vista *(Microsoft, n.d.)* this means that XP does not support it. That is why the recommendation is to get Windows 11 (64 bit) version at a price point of €177*(Microsoft, n.d.)*. This change also means that the software will need to be updated or reinstalled for Windows 11. This conflicts with the constraint (*Appendix A.1, Table 1*) and so before purchase the client must confirm that ProteinSim 4 can be changed to be compatible with Windows 11 and not hard work.

4.6 Auxiliary Devices

Moving on to the last part that a computer system needs to consider which is auxiliary devices. This section mostly covers peripheral devices like display, keyboard, mouse, UPS and other.

Firstly, the current display, mouse and keyboard are most likely too old to be used. The university will have to invest in buying some that fit goods with the needs. There is no easy way of recommending anything because some other information are required which consist of: the space available, where the PC will be, the requirements of the user regarding the display size, mouse and keyboard and other. It can be approximated that around €300- €400 needs to be spent.

Furthermore, the additional device that it would be recommended is to include an Uninterruptable Power Supply (UPS) in order to protect long calculations from failure caused by a power outage. UPS is a device that has batteries inside, when power from the main grid is out, UPS takes over and send the power to the computer. When the main grid power returns to normal, the UPS stops supplying power, recharges its batteries and the power to PC is taken from the grid *(Vertiv, n.d.)*. There is a type of UPS called multi-mode UPS, which means that the AC power can either go through the UPS directly or with a secure mode. If it goes with a secure mode it will regulate AC to protect the device from picks in the network grid *(Vertiv, n.d.)*. The recommendation here would be the CyberPower CP1500EPFCLCD which is rated at 1500VA and 900W. This UPS has a voltage regulator which will regulate the power input to keep the voltage at a steady safe level for the device *(CyberPower, n.d.)*. Also, the utilized power will be around 428W which will leave a lot of room for expansion or other devices possibly. The price for this UPS is around €393 *(CyberPower, n.d.)*.

There is not a lot of improvement in performance from these devices, but they improve reliability, efficiency and security of the device in terms of protecting the device from a power failure.

4.7 Solution Conclusion

In conclusion, Section 3 identified two major issues of the performance loss which is firstly a processor with a single working unit (core) which no software could work around and secondly a memory pattern that wastes performance. It was also discovered that the memory capacity was never a real problem since the largest matrices used today

are only 2.5% of the current memory. Section 4 proposed a replacement system that would address all of these issues. Table 15 below summarises the findings sized against the requirements(*see Appendix A.3, Table 4*).

Table 15

Conclusion of the current and new system

Component	Current	Recommended	Effect
Processor	1 core, 1 thread, 1MB L2, no L3	24 cores, 48 threads, 128MB L3	- Achieving true parallel execution, - Speedup by 11.2x, - AMAT of 2.67 lower,
Memory	4GB DDR, 2 channels, 6.4GB/s	64GB DDR4 ECC, 8 channels, 204.8GB/s	- ECC protecting data, - 32 times the bandwidth feeding all 24 cores,
Storage	80GB HDD, 150MB/s	2x 1TB NVMe in RAID 1, 7,450MB/s	- Now can survive one drive failure, - Lower load time of data by 50 times,
Graphics	Integrated GMA 900, shares RAM	NVIDIA RTX A400	- Display only, - Free up the RAM, - No effect on calculation,
System integration	LGA775, PCI, 430W supply, Windows XP	sWRX8, PCIe 4.0, 850W supply, Windows 11 Pro	- Allows the other components to reach their specification speeds,
Auxiliary devices	Existing peripherals	Mouse, Keyboard, UPS	- Protect work against power failure, - No effect on performance,

As it can be observed there is a lot of positive effects of the new system. The budget constraint was also met successfully *(see Table 16)*.

Table 16

Cost of the recommended system

Component	Selected Model	Cost
Processor	AMD Threadripper PRO 5965WX	€2,135
Memory	8x Kingston Server Premier 8GB DDR4-3200 ECC	€1,276
Storage	2 × Samsung 990 PRO 1TB	€570
Graphics	NVIDIA RTX A400	€205
Motherboard	ASUS Pro WS WRX80E-SAGE SE WIFI	€890
Cooling	Noctua NH-U14S TR4-SP3 with a second fan	€144
Power Supply	Corsair RM850x	€125

Case	Phanteks Enthoo Pro 2 Server Edition	€160
OS	Windows 11 Pro	€177
UPS	CyberPower CP1500EPFCLCD UPS	€393
Peripherals (display, keyboard, mouse)	To be selected	€300–€400
Total		**€6,375–€6,475**
Approved budget		**€7,000**
Remaining		**€525-€625**

As it can be noticed, the total cost of the whole new system is around €6,400 (as of the time of writing this report) which is around 90% of the approved budget. This means that there is still room if anything will need to be changed but not too big. All of the parts are standard components that can be bought so the 3-4 weeks timeframe can be achieved. Compatibility depends on the developer that needs to confirm that ProteinSim 4 can work on Windows 11.

Regarding the reuse of some parts that are in the current system, it would be very hard to reuse any of it. Firstly, this is because the components are too old and not compatible like the CPU or the motherboard. Secondly some of the components like cooling or power supply is just not enough for the new components. Lastly peripherals could be reused but they are probably also too old, and they are worn out and if there is budget it would be good to replace them too. The only thing that is kept is the old disk for archival of the old matrices and calculations for future, if it will be needed.

In conclusion the recommended system resolves all issues pointed out through this investigation within the approved budget for this client. The Expected improvement in performance is analysed in the next section, and the final improvement will be calculated in the program section later on.

5. Performance impact of the Solution

Previous sections established the recommended improvement of each components on its own. This section looks at the system as a whole and compares the

estimated/calculated improvement with real life independent benchmarks. The aim here is to prove the provided performance improvement and which once can only be calculated*(see Table 17)*.

Table 17

Comparison of the estimated performance improvement with benchmarks.

Component	Estimated improvement	Benchmark	Comparison	Effect on the Client
Processor	11.2x speedup with P=0.95 and 24 cores calculated with Amdahl's Law.	CPU Mark improvement from 240 to 65,823 so a 270x increase. *(PassMark Software, n.d.)*	Both the benchmark and the calculations show the same, direction of improvement but the size is not comparable. The estimate is lower because of the 5% celling but also because PassMark runs the test on all threads and newer architecture.	The client can finally run the software with multithreading in parallel so the matrix calculation will take way less time to complete which is the main fix of the problem.
Each thread in the processor	Not estimated.	Single thread rating improvement from 514 to 3318 so a 6.5x increase. *(PassMark Software, n.d.)*	Supports the claim that the performance gain comes from the architecture and not the clock speed.	There are parts of the software that cannot run in parallel. In this case even these parts will be able to run faster.
Cache	AMAT improvement from 209.92 to 78.61 cycles which is a 2.67x times lower.	Benchmark not found.	The hit rate is confirmed but the miss rate are worst-case assumption so they cannot be directly confirmed.	Each core will wait less for any data because most of it is stored in L1, L@ and L3 caches meaning that less trips will have to be done to the main memory.
Memory bandwidth	An improvement from 6.4to 204.8 GB/s so a 32x more at its peak.	The new system has 124GB/s measured in a benchmark *(HotHardware, 2022)*.	Confirmed direction only that the measures is two times less. The measures is about 29x more in the benchmarked environment.	Each core will receive a steady stream of matrices allowing each core not to seat idle waiting for memory but working all the required time.
Memory latency	An improvement from 15ns to 13.75ns which is about 8% lower latency.	No independent benchmark available because it cannot be calculated alone.	The gain is not very big and not confirmed by a benchmark, so the improvement rests on bandwidth and capacity.	Small effect because the memory was chosen for its bandwidth and not latency.
Storage transfer	54MB of data was loaded in 0.36s and now an improvement	Old drive was measured at around 55.6MB, result was	Estimates drive speed was a little bit slower, so the gain is about 137x.	Data saves and retrieves from storage almost instantly which makes that a huge improvement

	was made to 0.0072s so a 50x upgrade.	1s *(StorageReview, n.d.)*.		towards working with the software.
Storage access time	An improvement from 12.67ms to 0.045ms which is about 280x improvement.	Old drive was measured at 13.5ms which confirms the calculation. *(StorageReview, n.d.)*	The benchmarks confirms that the new drive is way faster in data access time.	Very small checkpoints, reads and writes happen with no stalls of the program allowing for much smoother operation.

Each improvement specified above address one part of the solution to the problem. As it can be seen the whole system is going to run faster and it will deliver much higher performance, but not only that. It is very important to say that the system, first time ever will enable parallelism that did not exist before. Besides the CPU there is only an improvement to the storage and memory in data transfer speeds and the capacity of both. This will allow for higher performance but also higher reliability and more processing capabilities.

Unfortunately, as it can be noticed some of the calculated data could not be confirmed by benchmarks because some of the data cannot be measured directly alone. The C++ program in section 9 tests some of these assumptions and will confirm the following declarations.

6. The Maintenance of the Solution

The recommended system has a very different maintenance profile from the current system. This is because the current system has a single core only and 89W TDP, a hard disk and a single fun whereas the recommendation has 280W processor, ECC, mirrored SSDs in RAID 1 and more. This means that the system not only needs to be cleaned and maintained but also there are software features like error logging and data redundancy that need to be controlled and maintained.

The cooling of the system is checked through temperature measurements. Firstly, a baseline needs to be obtained when the system is new and under a full load. Then each day the temperature is compared. If the temperature rises above the baseline maintenance needs to be done. This maintenance consists of cleaning fans, the case and

all filters from dust. Maintenance also is recommended to be conducted every 3 months in order to allow the system to use full operational power at all times because it prevents overheating which would slow down the CPU*(CORSAIR, n.d.)*. The thermal paste on the CPU needs to be replaced in the case of the cooler being removed but typically either way it needs to be replaced every two or three years because it loses its thermal properties *(darkFlash, 2025)*.

Moving from the physical maintenance to a software-based maintenance. The first one is the ECC, which correct memory-based single-bit errors invisibly although windows has a record system called WHEA. Windows Hardware Error Architecture which records the error detected by the ECC to its system event log*(Tittel, 2026)*. WHEA can also monitor memory pages that had errors earlier and if they exceed certain threshold it will try to take it offline. These errors can be used to predict new errors and as an ongoing practice, the error logs should be reviewed every month in order to check for frequent, repeating errors. If an error occurs when and it is fixed, it does not produce any harm, but if an error is repeated in the same module it should be addressed and the module should be replaced before an uncontrollable error occurs.

Now moving to disks and the RAID 1 setup. The health of both disks should be checked each month and the free space on both disks should be controlled frequently. NVMe drives report their own health data to the user such as the spare capacity, health percentage and critical warnings*(Samsung Electronics, 2022)*. Because of the RAID setup, if the first disk fails after some time it is very likely that the second may fail too because both are used int eh same way and the same amount which has to be kept in mind.

7. Alternative Architecture

The recommendation is bult around upgrading to a multi-core, multi-thread CPU but there is also another alternative for matrix multiplication. This alternative is to use GPU accelerated calculations in which the focus is on the GPU, and the GPU would do most of the calculation work. A CPU core is built in a way to finish one instruction stream

as quickly as possible through allocating all transistorise to caches, out of order logic and branch prediction. A GPU on the other hand, spends most of its transistors on a tone of small arithmetic units that at one time run the same instruction on different data *(Owens et al., 2008)*. This means that s single thread is much faster in a CPU, but a GPU has a much higher throughput. This means that matrix calculations would be expected to run faster on a GPU. Table 18 explains why the recommendation was still chosen to keep the CPU rather than a new GPU.

Table 18

Comparison of CPU and GPU-based accelerated architecture

Criterion	CPU Analysis	GPU Analysis	Reason for not choosing the GPU
Raw Parallel Throughput	24-core with 48 threads is enough to remove the bottleneck of a single core.	Has thousands of simple threads which process small portions of the program, which gives far higher peak throughput on regular calculations like matrix multiplication.	GPU wins with higher peak, but the client needs software to run faster not the highest possible peak.
Software	ProteinSim 4 already uses CPU threads, and it does not have to be changed.	ProteinSim 4 would have to be re-developed to accommodate GPU accelerated calculations.	Changing the software falls out of the given constraints meaning that a GPU is not suitable.
Single thread speed	Very high-speed threads, with low latency and super-fast caches.	Each thread is slow but relies on thousand other threads which will hide the latency.	The part of the program that cannot be run in parallel runs much slower on a GPU meaning that Amdahl's calculation would be wors.
Memory capacity	Up to 2TB of memory spread across 8 channels.	On top of the main memory there is also VRAM built into the GPU which has very high bandwidth.	GPU's memory is much smaller and insufficient to process matrices this size so they would have to be split into pieces.
Data movement	System memory holds data and there is no need to copy it.	Data inside the GPU is processed very quickly.	Data would have to pass though PCIe every time, inside CPU this problem does not exist because it has caches and direct memory connection.
Flexibility	Handles both regular and irregular operations and code very well	Handels only general and reparative operations well.	If the code has some irregular operations, the GPU cannot handle them well.

Risk and cost	Same software with costs fitting the budget.	Potentially larger speedup later but also may be outside of budget.	GU adds new, different hardware and software, larger power supply and software work development.

The table above shows that a GPU is better in some of the criteria like the parallel processing threads or memory speeds. On the other hand, CPU is a better choice for this because there is no need to change the software, it gets more work than with more complex tasks, it moves more data which is crucial and the cost and flexibility is better. Therefore, a CPU is a better option, but a GPU is a valid alternative, but it requires more time and work. It is also possible to extend the system later with a better GPU and then re-develop the ProteinSim 4 to accommodate GPU accelerated calculations.

8. Overview of the Testing Program/Application

The benchmarking program is a terminal-based C++ application that multiplies two square matrices, A and B, in four different ways which is used to analyse the improvement in parallelised processes by measuring the time needed to complete the calculations. The user enters the matrix size and the number of threads, and the program fills out the matrices with randomly generated values, but it has a fixed seed. This means that the generated data is identical across all calculations. These calculations are conducted in four different ways and then their speedup or Amdahl's Law is compared in order to see the effect of parallelism(further explanation can be found in *Section 9.1)*. Furthermore, matrix multiplication was chosen as the testing tool duo to the fact that the client is performing this kind of operations and so the closest point of reference is to use similar program to ProteinSim 4 which is capable of simulating similar direction of CPU and memory workload.

The purpose of the program is therefore to test, with measurements, whether the proposed solution of parallelizing the hardware removes the problem and the bottlenecks. The client suspected the RAM to be the issue, but the real problem was discovered as not enough cores and cache in the system which is tested through:

- Cores: running the same workload on single and muti threads and comparing the time of execution.
- Cache: running the same workload with and without cache optimized code showing how much more performance depends on cache then RAM.

The diagnosis is that the speedup will be higher with multithreading and the highest with cache optimised code confirmed in *Section 9.2 and 9.3*.

The program is simple on the architectural side but at the same time it is also complex in the implementation. When the program is run it asks the user for the input of the matrix size and the number of threads. Then the program based on the first input generates two square matrices of the selected size. There is a constraint of the seed and a range of values from zero to one, and the generated matrix values are between this two numbers. Furthermore, the program firstly does the multiplication on a single thread, then with multithread, then multithread with reorder matrix for faster memory access and finally multithread with the reordered matrix but also with matrices divided into smaller blocks to fit into cache spaces. Finally, for each of them the time of execution, the speedup and the proportion of program that is parallelized is printed out. Furthermore, in order to verify the correctness of each calculation, a checksum was created and if does not match, there is a failure printed out (detailed code explanation can be found in *Appendix C.1*)

9. Program Results Analysis

9.1 Testing Methodology

As already mentioned above, the methodology for this application is brought and vast. Firstly, it needs to be specified that all tests were run on a single machine, a MacBook Pro with a Silicon M4 Max Processor having 16 cores(12 performance and 4 efficiency cores), and with 64GB of RAM. The processor does not have SMT(simultaneous multithreading) meaning that a single core cannot run 2 threads at one time like the proposed CPU. The program was compiled with clang++26, and the program was run 5

times with no other background applications open and with the laptop plugged into power.

The parameters inputted to the program for testing were: the matrix size of 2000 and a thread count of 24 were used. Although the laptop used does not have that many cores and it cannot run these number of threads simultaneously, this is the closest it is possible to get to the real scenario. There is another option of running for example half the threads and half the matrix size but as tested, the speedup and parallel portion do not scale the same way duo to the equations and so the first option was chosen. Furthermore, matrices A and B, on each run has the same values duo to the set seed and number ranges from 0 to 1 but on each 5 tests the data is randomly generated but the size of each value stay constant.

On top of that, on each run 4 implementation of the same matrices multiplications were conducted, and each is an improvement on the previous one as described below:

1. Single-threaded (the baseline):
    - Simple three loops which iterates through the matricides and multiplies the column with each row as a baseline and later all checksums are compared to this result.
2. Multi-threaded:
    - The same algorithm as with single-threaded but the multiplication is divided for each thread and then at the end it connects together. This tests the effect of the number of threads.
3. Multi-threaded memory optimised (reordered loops/matrix A):
    - The same algorithm as with multi-threaded but with its loops being recorded so that matrix B is not read along the columns but rows making the memory reading sequential rather than jumping across to get the column. This is because the matrix is stored as firstly row, then second row all together so to get to the whole first column it needs to jump every x number of elements in each row. This means it reads its neighbour form the same cache line.
4. Multi-threaded memory optimised and cache-blocked (making smaller chunks of both matrices and computing at once):

- The same algorithm as with multi-threaded with memory optimized but matrices are computed as small 64 x 64 tiles at a time so that work stays in cache when it is reused and then discarded forever and never opened again because the block was already computed.

In here the single-threaded multiplication would represent the old system. This is because the old system had only one CPU. The second multiplication represents the recommended system although the number of cores on the testing machine is not the same as the recommended CPU. Third experiments shows that memory access pattern is very important in performance and finally the last one shows that having cache big enough to fit the matrices allows for also performance improvement.

The time for each multiplication is calculated only for the duration of the multiplication, so a time is started before and stopped right after the function (*see Appendix C.1*) so printing the results, inputting data or generating the matrix is not included. From this two values are calculated:

- The speedup = single thread time / multi thread time
- Parallel fraction (P) from Amdahl's Law = (1-1/S) / (1-1/N) and a value of 1 means that the whole programs runs in parallel.

The program was run five times in order to get the averages from all of them to get the most accurate results. Furthermore, also checksum was checked to make sure that all results are correct.

9.2 Raw Testing Data

Now moving to the raw data that is pulled when the program is run. In each experiment the checksums were not recorded because they all matched each time. The results from the 5 runs of the program can be found in *Appendix C.2*. Firstly, the time for each experiment was recorded in table 19 below.

Table 19

Execution times for each experiment and 5 runs with their averages

Version	Run 1	Run 2	Run 3	Run 4	Run 5	Average:
1. Single-threaded	6.0479	6.0260	6.0866	6.2904	6.05654	6.1015
2. Multi-threaded	0.5300	0.5671	0.5784	0.5612	0.5602	0.5594
3. Multi-threaded memory optimised	0.09805	0.1072	0.1008	0.09829	0.09998	0.1009
4. Multi-threaded memory optimised and cache-blocked	0.07539	0.09336	0.07870	0.07404	0.07259	0.0788

As it can be noticed, the average time of execution drops from 6.1s to 0.079s. The variation in each experiment for 5 runs is small meaning that the results are reliable. On top of that the change is big meaning that there is a noticeable difference, and the proposed system will work correctly in the big picture, but it will be further discussed in *Section 9.3*.

Now moving to the speedup that was given by the program for each of the experiments except the first one it acts as the baseline, see *Table 20*.

Table 20

Speedup for each experiment and 5 runs with their averages

Version	Run 1	Run 2	Run 3	Run 4	Run 5	Average:
2. Multi-threaded	11.41x	10.63x	10.52x	11.21x	10.81x	10.92x
3. Multi-threaded memory optimised	61.68x	56.24x	60.40x	64.00x	60.58x	60.58x
4. Multi-threaded memory optimised and cache-blocked	80.22x	64.55x	77.34x	84.96x	83.43x	78.10x

Regarding the speedup, the average changes from 10.92x which is relatively close to what I before of 11.3x. This is not exactly the same, but the difference is subtle. Regarding the other two experiments, it can be notices that the speed is way bigger, but this is not accurate 100% because the Amdahl's Law does not account for the change in software, but hardware which will be further discussed in *Section 9.3*.

Finally, the proportion that is parallel for each of the runs can be found in *Table 21*.

Table 21

Parallel proportion for each experiment and 5 runs with their averages

Version	Run 1	Run 2	Run 3	Run 4	Run 5	Average:
2. Multi-threaded	95.20%	94.53%	94.43%	95.04%	94.70%	94.78%
3. Multi-threaded memory optimised	102.66%	102.49%	102.62%	102.72%	102.63%	102.62%
4. Multi-threaded memory optimised and cache-blocked	103.05%	102.73%	103.00%	103.12%	103.10%	103.00%

The parallel portion on average in the second experiment is 94.78% which is also similar to the prediction of 95% from earlier, but it is not the same. This is because this proportion is calculated from the speedup, but it is very close to the prediction which will be further discussed in *Section 9.3*.

9.3 Benchmarking results analysis and Limitations

Now moving to the analysis of the raw data from *Section 9.2*, firstly the speedup was predicted at a level of 11.2x in *Section 5* for a 24-core CPU with a assumed parallel portion of P=0.95% with a celling of 20x. The measured speedup in the program is 10.92x (*Table 20*) which is around d97% of the prediction and the measured P was 94% accurate o almost exactly as the prediction. This means that the target of at least 10x was achieved. The gap comes from the difference in the cores because Mac chips cannot do 2 simultaneous threads on a single core. It also comes from the portion of the code that is parallel. This gap would not disappear if more cores would be added because of Amdahl's law meaning that the ceiling is stationary.

Moving on to the Average Memory Access Time(AMAT) which is the hit time added to miss rate multiplied with the miss penalty. The standard experiment in the program reads the matrix B down the column. This means that each read falls down a different cache line because memory saves by rows not columns. This also means that every read

of B is a miss because it needs to jump every time and when it comes back to multiply with the second row of A, the first values of the first column from B is long gone from the cache. The reordered version works by reordering columns into rows so one cache line (typically 64 bytes, i.e. 8 values) serves 8 consecutive reads and the miss rate falls sharply. Cache blocking goes even further by dividing the full matrix into multiple blocks of 64 x 64 tiles which can stay in cache for the period of computation and then when new tile is loaded the old one will never be reused. This is why reordering gave further 5.5x speedup and blocking a 1.28x more. The AMAT was estimated to be lower by 2.7x from larger cache. This cannot be measured in the program, but both values give the direction which means that the direction can be agreed.

Furthermore, Moore's Law state that extra transistors go into more cores rather than the speed of the clock. This is confirmed in this case because the clock speed rows only by around 1.4 while the multi-threaded memory optimized and cache locking gave around 77x in total.

Lastly there were also some limitations to this data dn the methodology. This can be found in *Table 22* below.

The Limitation	**Effect on the results**
The current hardware cannot be measured and used for measuring.	The estimated improvement is only an estimate from benchmarks and a program that simulates the real upgrade. There is no physical hardware to run it on and test it so no real data could be provided.
The testing machine is totally different then the proposed one, 16 vs 24 cores, 16 vs 48 threads, architecture is different and no simultaneous multithreading.	On a real 24-coe CPU the times and speedup would differ not a lot but there would be a difference.
24 threads were run on 16 core CPU	The threads run on the cores had to take turns so in reality the parallelised portion of the code was greater because the actual pert that was running in parallel was smaller than it really was.

The parallel proportion is only valid for the multi-threaded experiment not for the other two.	The P value is above 100% which is not a valid number. This is because program was changed and not hardware. This is also because the speedup includes the memory-access gain.
RAM could not be controlled between tests.	The effect of cache and cores on performance is larger but at the same time it indirectly says that RAM is not the bottleneck.
The software was not the real ProteinSim 4.	The real ProteinSim 4 can have a totally different structure with less parallel code meaning that speedup and P would differ.
Measurement distortion and errors.	There are few measurements which are totally different meaning that they will pull down the average down causing an inaccurate average value.

10. Conclusion

In summary, The Bellerman Molecular Research Centre though that their lack in performance was caused by too little RAM, as their IT department said. This investigation proved and showed otherwise, and the real reason was not the RAM itself. The largest matrix used today, take only about 2.4% of the current RAM capacity. The real bottlenecks are in the processor with single core on which ProteinSim 4 cannot run in parallel and the very little amounts of cache meaning that every calculation needs to be sent into the memory.

The solution to the system was a recommendation of a new hardware system. The main component is the AMD CPU Threadripper PRO 5965WX with 24 cores and 128 MB of L3 cache, 64 GB of 8-channel ECC memory, and NVMe storage in RAID 1. The estimated cost is about €6,400 which is roughly 90% of the €7,000 budget. Other parts are also recommended and all of that meets the constraints set at the beginning with the client.

The benchmark results prove that there will be an improvement by switching from series to parallel operation mode. The exact hardware improvement cannot be confirmed as the recommended and current systems are not available.

The future expansion is also possible. The hardware leaves a lot of room for extra memory, storage and performance improvement. On top of that there is an expectation to use larger matrices for which the recommended system is scaling for.

11. Reference

1. Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Conference Proceedings*, *30*, 483–485.
2. AMD. (n.d.). *AMD Ryzen Threadripper PRO 5965WX drivers and downloads*. https://www.amd.com/en/support/downloads/drivers.html/processors/ryzen-threadripper-pro/ryzen-threadripper-pro-5000wx-series/amd-ryzen-threadripper-pro-5965wx.html
3. Anderson, M. (2025, May 25). *The cache clash: L1, L2, and L3 in CPUs*. Medium. https://medium.com/@mike.anderson007/the-cache-clash-l1-l2-and-l3-in-cpus-2a21d61a0c6b
4. ASUS. (n.d.). *Pro WS WRX80E-SAGE SE WIFI*. https://www.asus.com/motherboards-components/motherboards/workstation/pro-ws-wrx80e-sage-se-wifi/
5. Bjorn3D.com Reviewer. (2003, March 10). *Antec TruePower 430*. Bjorn3D.com. https://bjorn3d.com/2003/03/antec-truepower-430/
6. Chen, Y., & Oyelami, B. (2024). *Evolution, challenges, and optimization in computer architecture: The role of reconfigurable systems*. arXiv. https://arxiv.org/pdf/2412.19234
7. Cloud, R. (n.d.). *CPU clock speed importance – why GHz impacts performance — Modern Server Solutions | Redfox Cloud*. Redfox Cloud. https://www.redfoxcloud.com/en/article/cpu-clock-speed-importance

8. Connatser, M. (2023, June 14). *What is a process node?* XDA. https://www.xda-developers.com/process-node/
9. Corsair. (n.d.). *MT/s vs MHz explained*. https://www.corsair.com/us/en/explorer/diy-builder/memory/mts-vs-mhz-explained/
10. Corsair. (n.d.). *RMx Series RM850x fully modular power supply* (CP-9020270-NA). Retrieved October 6, 2026. https://www.corsair.com/us/en/p/psu/cp-9020270-na/rmx-series-rm850x-fully-modular-power-supply-cp-9020270-na
11. CORSAIR. (n.d.). *How to clean your PC, and how often you should do it*. https://www.corsair.com/us/en/explorer/diy-builder/how-tos/how-to-clean-your-pc-and-how-often-you-should-do-it/
12. CPU-Monkey. (2025). *Intel Xeon w7-3455 - Benchmarks, specifications, user reviews & CPU comparisons*. Retrieved October 6, 2026. https://www.cpumonkey.com/en/cpu-intel_xeon_w7_3455
13. Crucial. (n.d.). *What is error correcting code (ECC) memory?* https://www.crucial.com/articles/about-memory/what-is-ecc-memory
14. CS 61C Course Staff. (n.d.). *Average memory access time (AMAT)*. UC Berkeley CS 61C Course Notes. https://notes.cs61c.org/content/caches-intro/amat/
15. CyberPower. (n.d.). *CP1500EPFCLCD*. Retrieved October 6, 2026. https://www.cyberpower.com/eu/en/produc t/sku/cp1500epfclcd
16. darkFlash. (2025, December 2). *How often should you replace thermal paste? The ultimate guide for CPUs and GPUs*. https://www.darkflash.com/article/how-often-should-replace-thermal-paste
17. Demmel, J. (2016). *CS267 lecture 2: Single processor machines: Memory hierarchies and processor features*. University of California, Berkeley. https://people.eecs.berkeley.edu/~demmel/cs267_Spr16/Lectures/lecture02_memhier_jwd16_4pp.pdf

18. Djidjev, H. N., Hahn, G., Mniszewski, S. M., Negre, C. F. A., Niklasson, A. M. N., & Sardeshmukh, V. B. (2016). *Graph partitioning methods for fast parallel quantum molecular dynamics*. arXiv. https://arxiv.org/pdf/1605.01118

19. GeeksforGeeks. (n.d.). *Direct memory access (DMA) controller in computer architecture*. https://www.geeksforgeeks.org/direct-memory-access-dma-controller-in-computer-architecture/

20. Hard Disk Sentinel. (n.d.). *Technical details for hard disk drive SEAGATE ST380817AS*. Retrieved October 6, 2026. https://www.hdsentinel.com/storageinfo_details.php?lang=en&model=SEAGATE%20ST380817AS

21. Hennessy, J. L., & Patterson, D. A. (2019). *Computer architecture: A quantitative approach* (6th ed.). Morgan Kaufmann.

22. Hinum, K. (n.d.). *Intel Graphics Media Accelerator 900*. Notebookcheck. https://www.notebookcheck.net/Intel-Graphics-Media-Accelerator-900.2178.0.html

23. HMS Networks. (2024, July 29). *What is CAN arbitration and how does this work?* HMS Networks Tech Blog. https://www.hms-networks.com/tech-blog/blogpost/hms-blog/2024/06/18/what-is-can-arbitration-and-how-does-this-work

24. HotHardware. (2022). *AMD Threadripper Pro 5000WX review* (page 3). https://hothardware.com/reviews/amd-threadripper-pro-5000wx-review?page=3

25. Icecat. (n.d.). *Kingston Technology ValueRAM 1GB 400MHz DDR non-ECC CL3 (3-3-3) DIMM memory module, 1 x 1 GB, 184-pin DIMM (KVR400X64C3A/1GB)*. https://icecat.biz/p/kingston+technology/kvr400x64c3a-1gb/valueram-memory+modules-1gb+400mhz+ddr+non-ecc+cl3+-3-3-3-+dimm-81083.html

26. Intel Corporation. (2004, September). *Intel 915G/915GV/910GL Express chipset graphics and memory controller hub (GMCH) white paper* (Document No. 301671-

003). https://theretroweb.com/chip/documentation/30167103-6696452e8868f295650854.pdf

27. IONOS. (2023, January 30). *What is RAID 1?* IONOS Digital Guide. https://www.ionos.com/digitalguide/server/security/raid-1/
28. Kingston. (n.d.). *Server Premier memory for data centers*. Retrieved October 6, 2026. https://www.kingston.com/unitedkingdom/en/memory/server-premier
29. Kingston. (2025). *What is CAS latency? CL and RAM timings explained*. https://www.kingston.com/en/blog/gaming/cas-latency-cl-ram-timing-explained.
30. Lau, J. (2025, April 16). *What is a cache? And why does clearing it fix things?* https://zapier.com/blog/what-is-a-cache/
31. Lenovo. (n.d.). *What is Windows XP?* Lenovo Glossary. https://www.lenovo.com/us/en/glossary/windows-xp/
32. *Matlantis*. (2025, July 7). *Molecular Dynamics Simulations for materials and molecule discovery - from fundamental to emerging trends*. MATLANTIS. https://matlantis.c om/en/resources/blog/md-intro/
33. McKee, S. A. (2011). Memory wall. In D. Padua (Ed.), *Encyclopedia of parallel computing*. Springer. https://link.springer.com/rwe/10.1007/978-0-387-09766-4_234
34. Memory Systems Authority. (n.d.). *Memory channel configurations: Single, dual, quad-channel explained*. https://memorysystemsauthority.com/memory-channel-configurations
35. Microsoft. (n.d.). *Overview of NDIS MSI-X*. Microsoft Learn. https://learn.microsoft.com/en-us/windows-hardware/drivers/network/ndis-msi-x
36. Microsoft. (n.d.). *Windows 11 Pro (download)*. Microsoft Store. Retrieved October 6, 2026. https://www.microsoft.com/en-us/d/windows-11-pro/dg7gmgf0d8h4

37. Moore, G. E. (1965). Cramming more components onto integrated circuits. *Electronics*, *38*(8), 114–117.

38. Noctua. (n.d.). *NH-U14S TR4-SP3: Features*. Retrieved October 6, 2026. https://www.noctua.at/en/products/nh-u14s-tr4-sp3/specifications

39. NVIDIA. (n.d.). *NVIDIA RTX A400*. Retrieved October 6, 2026. https://www.nvidia.com/en-us/products/worksta tions/rtx-a400/

40. OEMdrivers. (n.d.). *Realtek RTL8110SC-GR*. https://oemdrivers.com/realtek-rtl8110sc-gr

41. Owens, J. D., Houston, M., Luebke, D., Green, S., Stone, J. E., & Phillips, J. C. (2008). GPU computing. *Proceedings of the IEEE, 96*(5), 879–899. https://doi.org/10.1109/JPROC.2008.917757

42. PassMark Software. (n.d.). *AMD Ryzen Threadripper PRO 5965WX vs Intel Pentium 4 2.80GHz*. CPU Benchmarks. Retrieved October 6, 2026 https://www.cpubenchmark.net/compare/4768vs1072/AMD-Ryzen-Threadripper-PRO-5965WX-vs-Intel-Pentium-4-2.80GHz

43. Patterson, D. A., Gibson, G., & Katz, R. H. (1988). A case for redundant arrays of inexpensive disks (RAID). *ACM SIGMOD Record*, *17*(3), 109–116. https://doi.org/10.1145/971701.50214

44. PCPartPicker. (n.d.). *Noctua A15 PWM 140 mm fan (NF-A15 PWM)*. Retrieved October 6, 2026. https://pcpartpicker.com/product/znPfrH/noctua-case-fan-nfa15pwm

45. Phanteks. (n.d.). *Enthoo Pro 2 Server Edition*. Retrieved October 6, 2026. https://phanteks.com/product/enthoo-pro-2-server-edition-tg/

46. Puget Systems. (n.d.). *ASUS Pro WS WRX80E-SAGE SE WIFI*. Retrieved October 6, 2026. Retrieved October 6, 2026. https://www.pugetsystems.com/parts/Motherboard/ASUS-Pro-WS-WRX80E-SAGE-SE-WIFI-13932/

47. Saitech Inc. (n.d.). *Intel C91249-002 socket-478 3-pin Pentium IV 2.5-inch aluminium heatsink cooling fan*. eSaitech. https://esaitech.com/products/intel-socket-478-3-pin-pentium-iv-2-5-inch-aluminium-heatsink-fan-c91249-002

48. Samsung Electronics. (2022). *Samsung Electronics unveils high-performance 990 PRO SSD optimized for gaming and creative applications*. Retrieved October 6, 2026. https://news.samsung.co m/global/samsung-electronics-unveils-high-performance-990-pro-ssd-optimized-for-gaming-and-creative-applications

49. Samsung Electronics. (n.d.). *990 PRO PCIe 4.0 NVMe M.2 SSD MZ-V9P1T0B/AM*. https://www.samsung.com/us/memory-storage/nvme-ssd/990-pro-pcie-4-0-nvme-ssd-2tb-sku-mz-v9p1t0b-am/

50. Schmid, P. (2004, February 1). *Intel Pentium 4 aka Prescott in detail.* Tom's Hardware. https://www.tomshardware.com/reviews/intel,751-3.html

51. ScienceDirect Topics. (n.d.). *Molecular dynamics simulation*. In *ScienceDirect Topics*. https://www.sciencedirect.com/topics/computer-science/molecular-dynamics-simulation

52. Spiryn, J. (n.d.). *What Is RISC and CISC Architecture & Their Differences*. https://conclusive.tech/glossary/what-is-risc-and-cisc-architecture-their-differences/

53. Stone, J. E., Phillips, J. C., Freddolino, P. L., Hardy, D. J., Trabuco, L. G., & Schulten, K. (2007). Accelerating molecular dynamic simulation on graphics processing units. *Journal of Computational Chemistry*, *28*(16), 2618–2640. https://pmc.ncbi.nlm.nih.gov/articles/PMC2724265/

54. StorageReview. (n.d.). *Seagate Barracuda 7200.7 review*. https://www.storagereview.com/review/seagate-barracuda-7200-7

55. Submillisecond. (2026, August 08). *L1 cache*. Submillisecond Glossary. https://www.submillisecond.com/glossary/hardware/l1-cache

56. TECH-R-US Store. (2026). *Kingston KSM32RS8/8MR 8GB DDR4 3200MHz PC4-25600 ECC Reg RDIMM server memory RAM* [Listing]. eBay. Retrieved October 6, 2026. https://www.ebay.com/itm/820164821011

57. The Retro Web. (n.d.). *Intel D915GAG (Augsburg)*. https://theretroweb.com/motherboards/s/intel-d915gag-augsburg

58. Tittel, E. (2026, August 3). *Understanding Windows Hardware Error Service*. https://www.edtittel.com/blog/understanding-windows-hardware-error-service.html

59. Tom's Hardware. (2022). *Shed a tear for HEDT: AMD's official Threadripper Pro pricing marks the end of an era*. Retrieved October 6, 2026. https://www.tomshardware.com/news/shed-a-tear-for-hedt-amds-threadripper-pro-pricing-marks-the-end-of-an-era

60. Tutorialspoint. (n.d.). *Operating system - Virtual memory*. https://www.tutorialspoint.com/operating_system/os_virtual_memory.htm

61. Vertiv. (n.d.). *What is a UPS?* https://www.vertiv.com/en-us/about/news-and-events/articles/educational-articles/what-is-a-ups/

62. Walilko, A. (2025, May 5). *What Are CPU Cores vs Threads?* Liquid Web. https://www.liquidweb.com/blog/difference-cpu-cores-thread//

63. Western Digital. (n.d.). *HDD vs SSD: Choosing the right drive for your data*. https://www.westerndigital.com/solutions/hdd-vs-ssd.

64. *What is Clock Speed & Why Does It Matter? | Lenovo NZ*. (n.d.). https://www.lenovo.com/nz/en/glossary/what-is-clock-speed/?orgRef=https%253A%252F%252Fwww.google.com%252F&srsltid=AU7gw4XdH1cJS9PQwPvXrNYH6M8RR01bP3CuQtiImne53FhDnIqCZkkD

65. *What is a Memory Channel? RAM Performance Explained*. (n.d.). https://pcbstore.com.bd/glossary/memory-channels

Table 0

AI Acknowledgment table

AI use Category	Tools	How AI tools was used in this assignment
Inspiration	ChatGPT-4.0	Key topics, and word search related to learning objectives and outcomes.
Learning	ChatGPT-4.0 & Claude Sonnet 5.0	Search related to learning new hardware devices and research key new ideas.

12. Appendix

12.1 A.1 – Clients Constraints and Current System

Table 1

Constraints of Bellerman Molecular Research Centre

Constraint	Details	Measure
Budget	The maximum budget that is approved under the current research grant that can be justified for the expansion of the laboratory (excluding VAT).	€7,000
Time	The new hardware system needs to be completed before the grant deadline and there is no place for trial-and-error fixes or delays.	3-4 weeks
Compatibility	Any new system must be compatible with the pre-existing software and hardware in the university lab. For example, the calculation software ProteinSim 4.	-
Reuse	Any existing hardware components like storage, peripherals, etc. should be reused and retained to minimize expenditure.	-

Table 2

Specifications of the current system

Component	Model/Type	Parameters	Description
Processor/Central Processing Unit(CPU) *(Schmid, 2004)*	Intel Pentium 4 2.80A GHz "Prescott"	• Clock speed: 2.8GHz • Cores: 1 • Threads: 1 • Process node: 90nm • Hyper-Threading: Not supported • TDP: 89W	Single core, single threaded with no support for multi or hyper threading. This CPU is limited to executing only one instruction stream at a time. In this case no software is capable of achieving true parallelism or multi operation execution regardless of the design *(Hennessy & Patterson, 2019)*.
Level 1 (L1) Cache *(Schmid, 2004)*	Prescott on-die L1	• Data cache: 16KB • Trace cache: 12K micro-ops	L1 cache in that size is too small to hold even the smallest fraction of a single simulation matrix let alone three at the same time. This results in frequent cache misses during matrix operations.
Level 2 (L2) Cache *(Schmid, 2004)*	Prescott unified L2	• Size: 1MB • Type: Unified	L2 cache that is unified means that it stores both instructions and data together in the same place. This one is still far too little to store a matrix needed for calculations and because it is unified, instructions and data compete for the same space.
Level 3 (L3) Cache *(Schmid, 2004)*	Not available	Not available	In this CPU there is no level 3 cache meaning that every L1 and L2 cache miss results in a long process of going to the main memory rather than a shared cache.
Primary Memory/Random Access memory(RAM) *(Icecat, n.d.)*	4x Kingston ValueRAM KVR400X64C3A/1G	• Capacity: 4x 1GB so 4GB total • Type: DDR1 SDRAM • Speed: classified as PC-3200 so 400MT/s • Latency: CL3	A very outdated, slow and with high latency component. It was correct and easy to assume that this may be the issue of the software issues, but it is most likely not the cause as in one program run with today's biggest matrix size (up to 2,000x2,000) at most 2.5% of

			its capacity is used *(see Appendix A.2, Table 3)*.
Secondary memory/Storage *(Hard Disk Sentinel, n.d.)*	Seagate Barracuda 7200.7 ST380817AS	• Capacity: 80GB • Interface: SATA 150 • Speed: 7200RPM • Buffer: 8MB • Average seek time: 8.5ms • Power: • Seek: 12.8W • Idle: 7.2W	Old memory storage which has a very low-capacity comparing to modern capacities, low read and write speed and has a small buffer. Although this seems like an issue at the end it may not be the biggest one here.
Graphical Processing Unit(GPU) *(Hinum, n.d.)*	Intel GMA 900 (integrated, through the 915G chipset)	• Core clock speed: 333MHz • Memory: no dedicated VRAM so shares RAM with the system • Pipelines: 4 pixel / 0 vertex shaders • No T&L (Transform and Lighting) support	Offers no meaningful acceleration to CPU based processing because of its lack of its own VRAM memory and rather sharing the RAM.
Auxiliary Input and Output Devices (I/O)	• Intel ICH6 Southbridge *(Intel Corporation, 2004)*	• USB: 2.0 Hi-Speed USB	Because of these auxiliary devices data transfer and read/write operation from outside are limited in speed. This also restricts how quickly the data can be taken from the computer system and shared.
	• Realtek RTL8110SC *(OEMdrivers, n.d.)*	• Onboard Ethernet 10/100/1000Mbps	
Power Supply Unit(PSU) *(Bjorn3D.com Reviewer, 2003)*	• Antec TruePower 430	• Maximum output power: 430W • Efficiency: >68% • Cooling: Dual fan	Sufficient for the current system. It support all components and has a little bit of power left in the buffer. If the CPU or any other component changes it does not have enough overhead to support it.
Cooling *(Saitech Inc., n.d.)*	Stock Intel cooling fan (C91249-002)	• Single fan, 2.5 inch	Enough for the current single core set up but when upgraded with multiple core,

		• Design: Aluminium heatsink with copper core • Connection: 3-pin • Sockets support: 478, P4 up to 3.40GHz	it will not be able to support the heat transfer.
Motherboard/Chipset *(The Retro Web, n.d.)*	Intel D915GAG	• Chipset: Intel 915G • Socket: LGA775 • Memory support: 4× DDR1 up to 4GB • I/O: • 8× USB 2.0, • 4× SATA, • 1× RJ-45 LAN • Expansion: • 2× PCI, • 1× PCIe x1, • 1× PCIe x16	This outdated motherboard has no problem supporting current system but with an upgrade the PCI bandwidth would be a problem in data transfer.
Operating System (OS) *(Lenovo, n.d.)*	Windows XP (Professional Edition)	• Minimum RAM: 64MB • Recommended RAM: 128MB • Release: 2001	A lightweight very outdated system compared to modern times but requiring only small fraction of the memory allowing for more memory being used by the program and calculations.

12.2 A.2 – Matrix size Calculation and Interpretation

Table 3

Matrix size calculations of increasing problem size

Size of the Matrix (N)	Elements (N^2)	Memory per Matrix (N^2 x 8 bytes)	Total memory (3 x Memory per Matrix)	% of the current RAM utilized
1,000 x 1,000	1,000,000	1,000,000 x 8 = 8,000,000 bytes = 8MB	8MB x 3 = 24MB	24MB / 4,000MB = 0.60%

1,500 x 1,500	2,250,000	2,250,000 x 8 = 18,000,000 bytes = 18MB	18MB x 3 = 54MB	54MB / 4,000MB = 1.35%
2,000 x 2,000	4,000,000	4,000,000 x 8 = 32,000,000 bytes = 32MB	32MB x 3 = 96MB	96MB / 4,000MB = 2.40%
10,000 x 10,000	100,000,000	100,000,000 x 8 = 800,000,000 bytes = 800MB	800 x 3 = 2,400MB	2,400 /4,000MB = 60.0%

The first three rows represent the current matrix sizes used. The last row represents what the client expects to be calculating in near future.

As it can be observed, with the matrices used today the maximum used capacity is around 2.4% which shows that memory size isn't the problem although with the future matrices the memory usage as calculated is around 60% of the capacity which can be a future problem. Adding extra buffer for the OS, background services and other applications, the memory usage may increase by few hundred megabytes. The operating system itself recommends a 128MB which is a small fraction of the 4GB *(Lenovo, n.d.)*.

Performance is not only limited by the core count but also the CPU caches. With only 16KB of L1, 1MB of unified L2 and no L3 caches, almost every operation on the matrix is forced to go to and from the main memory rather than staying close to the CPU.

12.3 A.3 – New system requirements

Table 4

Requirements of the new system

Requirements	Value	Basics
Matrix size today	Up to 2,000 × 2,000	Matrix size specified by the laboratory from Appendix A.2, Table 3
Matrix size expected in the future	10,000 × 10,000	Client's assumption for future
Largest working size et	2,400MB	3 x N^2 x 8 bytes (*Appendix A.2, Table 3*)
Largest arithmetic size set	2 x 10^{12} floating-point operations	Calculated in Section 4.1
Plain data moved with plain loops	About 8TB	Calculated in Section 4.2
Parallel speedup	At least 10×	Design target, an assumption: about half of the 20× ceiling at P = 0.95
Budget	€7,000 excluding VAT	Appendix A.1, Table 1

The speedup target is an assumption since the client did not specify it, as 10x that is ambiguous but achievable

12.4 C.1 - Source Code Listing

Figure 3

Image of the general functions and single thread code from the main file

```
1   // Main application implementation together
2
3   #include "mat_operations.h" //Including the header file for matrix operations
4   #include <iostream> //Including the input-output stream library for console input and output
5   #include <chrono> //Including the chrono library for measuring time
6   #include <iomanip> //Including the iomanip library for formatting output
7
8   //Structure for returning data from each experiment run
    You, 20 hours ago | author (You)
9   struct RunResult
10  {
11      double time;
12      double checksum;
13  };
14
15  //Function to calculate the checksum of a matrix
16  double ChecksumCheck(const MatrixNa& C)
17  {
18      double checksum = 0.0;
19      for (double value : C)
20      {
21          checksum += value;
22      }
23      return checksum;
24  }
25
26  //Function for single thread multiplication
27  RunResult SingleThreadedMatrixMultiplication(const MatrixNa& A, const MatrixNa& B, MatrixNa& C, std::size_t size)
28  {
29      auto starttime = std::chrono::steady_clock::now(); //Start of the timer
30      MatrixMultiplySingle(A, B, C, size); //Running the function for single thread multiplication
31      auto endtime = std::chrono::steady_clock::now(); //Stop of the timer
32
33      std::chrono::duration<double> DurationTimeSingle = endtime - starttime; //Calculating the duration of the function run
34
35      double checksum = ChecksumCheck(C); //Sum of all elements of C: a quick fingerprint that every later version must reproduce
36
37
38      std::cout << std::fixed << std::setprecision(8);
39      std::cout << "\n";
40      std::cout << "---------------Single-threaded(Sequential)--------------" << "\n";
41      std::cout << "Time: " << DurationTimeSingle.count() << " s\n";
42      std::cout << "Checksum: " << checksum << "\n";
43      std::cout << "--------------------------------------------------------" << "\n";
44      std::cout << "\n";
45
46      return RunResult(DurationTimeSingle.count(), checksum); //Returning the time and checksum of the single-threaded run
47  }
48
```

At the beginning the first 4 lines are used to include certain files or packages to this main file. The first one includes the header file with all the base functions specified (see Figure 7). Then the input and output package is included in order to allow terminal inputs and outputs, the chrono which is used to measure the time for each experiment and finally a library used to format the output. Moving on a struct is defined which is a user created grouped data structure used to group data together. This also later allows for retrieving from a function both the time and the checksum in one go. Then the function in line 16 returns a double and it is used to calculate the checksum of the matrix C for each experiment. Lastly a function used to call the single thread multiplication and does few other things around it. Firstly, it sends through Matrix A and B from the main function below. Then the first thing is to start the timer, call the function and stop the timer and then calculate the difference. This will run the multiplication and calculate the time difference so the time it took for the multiplication to complete. Then a checksum is calculated by sanding though the calculated Matrix C to the function from line 16. Then the output to the terminal is created with cosmetics. Finally, the function returns wherever it is called (in the main function) the two variables so the checksum and the time with the use of variable from line 9.

Figure 4

Image of the multi thread and reorder with multi thread code from the main file



```
//Function for multi thread multiplication
RunResult MultiThreadedMatrixMultiplication(const MatrixNa& A, const MatrixNa& B, MatrixNa& C, std::size_t size, std::size_t threads)
{
    auto starttime = std::chrono::steady_clock::now(); //Start of the timer
    MatrixMultiplyMulti(A, B, C, size, threads); //Running the function for multi thread multiplication
    auto endtime = std::chrono::steady_clock::now(); //Stop of the timer

    std::chrono::duration<double> DurationTimeMulti = endtime - starttime; //Calculating the duration of the function run

    double checksum = ChecksumCheck(C); //Same fingerprint as the single-thread version, the two must match

    std::cout << std::fixed << std::setprecision(8);
    std::cout << "----------------Multi-threaded(Parallel)----------------" << "\n";
    std::cout << "Time: " << DurationTimeMulti.count() << " s\n";
    std::cout << "Checksum: " << checksum << "\n";
    std::cout << "------------------------------------------------------" << "\n";
    std::cout << "\n";

    return RunResult(DurationTimeMulti.count(), checksum); //Returning the time and checksum of the multi-threaded run
}

//Function for multi thread multiplication with memory-optimised (reordered) loops
RunResult MultiThreadedReorderedMatrixMultiplication(const MatrixNa& A, const MatrixNa& B, MatrixNa& C, std::size_t size, std::size_t threads)
{
    auto starttime = std::chrono::steady_clock::now(); //Start of the timer
    MatrixMultiplyMultiReordered(A, B, C, size, threads); //Running the function for multi thread multiplication with memory-optimised (reordered) loops
    auto endtime = std::chrono::steady_clock::now(); //Stop of the timer

    std::chrono::duration<double> DurationTimeReordered = endtime - starttime; //Calculating the duration of the function run

    double checksum = ChecksumCheck(C); //Same fingerprint as the single-thread version, they must match

    std::cout << std::fixed << std::setprecision(8);
    std::cout << "------Multi-threaded, memory-optimised(Sequential)------" << "\n";
    std::cout << "Time: " << DurationTimeReordered.count() << " s\n";
    std::cout << "Checksum: " << checksum << "\n";
    std::cout << "------------------------------------------------------" << "\n";
    std::cout << "\n";

    return RunResult(DurationTimeReordered.count(), checksum); //Returning the time and checksum of the multi-threaded run with memory-optimised
    (reordered) loops
}
```

In this image there are two functions, one for the multiplication with multi-threading and one with also reordering of the loops so memory optimized access. Both do the same exact things as the one on figure 3 before just with the difference of what is printed and which calculation function is called.

Figure 5

Image of the cache blocking multi thread and laws printing code from the main file

```
//Function for multi thread multiplication with memory-optimised (reordered) loops and cache blocking
RunResult MultiThreadedReorderedBlockedMatrixMultiplication(const MatrixNa& A, const MatrixNa& B, MatrixNa& C, std::size_t size, std::size_t threads)
{
    auto starttime = std::chrono::steady_clock::now(); //Start of the timer
    MatrixMultiplyMultiReorderedBlocked(A, B, C, size, threads); //Running the function for multi thread multiplication with memory-optimised (reordered) loops and cache blocking
    auto endtime = std::chrono::steady_clock::now(); //Stop of the timer

    std::chrono::duration<double> DurationTimeReorderedBlocked = endtime - starttime; //Calculating the duration of the function run

    double checksum = ChecksumCheck(C); //Same fingerprint as the single-thread version, they must match

    std::cout << std::fixed << std::setprecision(8);
    std::cout << "------Multi-threaded and cache-blocked(Sequential)------" << "\n";
    std::cout << "Time: " << DurationTimeReorderedBlocked.count() << " s\n";
    std::cout << "Checksum: " << checksum << "\n";
    std::cout << "-----------------------------------------------------" << "\n";
    std::cout << "\n";

    return RunResult(DurationTimeReorderedBlocked.count(), checksum); //Returning the time and checksum of the multi-threaded run with memory-optimised (reordered) loops and cache blocking
}

//Prints the speedup and the laws of one multi-threaded version compared with the single-thread run
void PrintLaws(const char* name, double secondsSingleThread, double secondsMultiThread, std::size_t threads)
{
    double speedup = CalculateSpeedup(secondsSingleThread, secondsMultiThread); //Calculating the speedup of the multi-threaded run compared to the single-thread run
    double parallelFraction = CalculateAmdahlParallelFraction(speedup, threads); //Calculating the parallel fraction P of the program measured from the speedup (Amdahl's law solved for P)

    std::cout << std::fixed << std::setprecision(2);
    std::cout << "----------" << name << " (" << threads << " threads)----------" << "\n";
    std::cout << "Single-thread time: " << secondsSingleThread << " s\n";
    std::cout << "Multi-thread time: " << secondsMultiThread << " s\n";
    std::cout << "Speedup vs single-thread: " << speedup << "x\n";
    std::cout << "Parallel fraction P (Amdahl): " << parallelFraction << "%\n";
    std::cout << "-----------------------------------------------------" << "\n";
    std::cout << "\n";
}

```

In the image above, there is a code for a function which is the same as in Figure 3 but for multi-threading with cache blocking. Furthermore, there is also another function which does not return anything, and it is used to print out the speedup and parallel portion, so the laws used in this program. Firstly, it calls the speedup calculation function discussed further on and it sends through the single and multi-thread time and saves the output. Then another function from the same file is called to calculate the parallel portion by sending the speedup and the number of threads. All of these variables will be taken from the place where the PrintLaws is called so the main function. Finally, it prints out with cosmetic settings the results.

Figure 6

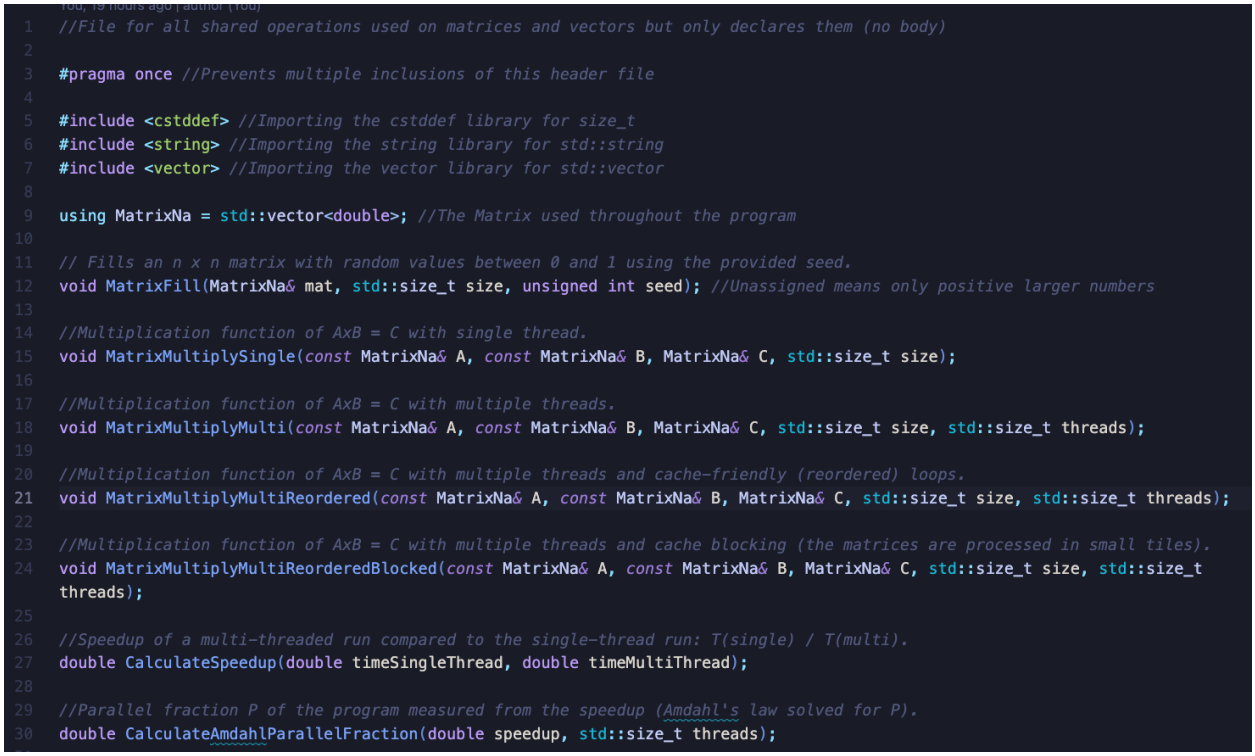
Image of main function code from the main file

```
128 //Main function
129 int main()
130 {
131     //Asking user for input on the size of the matrix and checking if valid
132     std::size_t size = 0;
133     std::cout << "Enter matrix size (e.g. 1000, 1500 or 2000): ";
134     std::cin >> size;
135     if (size == 0)
136     {
137         std::cout << "Matrix size must be greater than 0. Assuming 1000\n";
138         size = 1000;
139     }
140
141     //Asking user for input on the size of the matrix and checking if valid
142     std::size_t threads = 0;
143     std::cout << "Enter number of threads: ";
144     std::cin >> threads;
145     if (threads == 0)
146     {
147         std::cout << "Number of threads must be greater than 0. Assuming 1\n";
148         threads = 1;
149     }
150
151     //Calling the function to create the matrices
152     MatrixNa A, B, C;
153     MatrixFill(A, size, 1);   // seed 1 for A
154     MatrixFill(B, size, 2);   // seed 2 for B
155
156     //Calling thread calculation function
157     RunResult SingleThreadedData = SingleThreadedMatrixMultiplication(A, B, C, size);
158     RunResult MultiThreadedData = MultiThreadedMatrixMultiplication(A, B, C, size, threads);
159     RunResult MultiThreadedReorderedData = MultiThreadedReorderedMatrixMultiplication(A, B, C, size, threads);
160     RunResult MultiThreadedReorderedBlockedData = MultiThreadedReorderedBlockedMatrixMultiplication(A, B, C, size, threads);
161
162     //Printing if all results are the same
163     if (SingleThreadedData.checksum == MultiThreadedData.checksum && SingleThreadedData.checksum == MultiThreadedReorderedData.checksum &&
        SingleThreadedData.checksum == MultiThreadedReorderedBlockedData.checksum)
164     {
165         std::cout << "\n";
166         std::cout << "CHECKSUM MATCHES. THE RESULTS ARE CONSISTENT." << "\n";
167         std::cout << "\n";
168     }     You, yesterday • adding matrix multiplication with mutli threadi…
169     else
170     {
171         std::cout << "\n";
172         std::cout << "CHECKSUMS DO NOT MATCH. THERE IS AN INCONSISTENCY IN THE RESULTS." << "\n";
173         std::cout << "\n";
174     }
175
176     //Laws: each multi-threaded version compared with the single-thread run
177     std::cout << "-------------------------LAWS----------------------------" << "\n";
178     PrintLaws("Multi-threaded", SingleThreadedData.time, MultiThreadedData.time, threads);
179     PrintLaws("Memory-optimised", SingleThreadedData.time, MultiThreadedReorderedData.time, threads);
180     PrintLaws("Cache-blocked", SingleThreadedData.time, MultiThreadedReorderedBlockedData.time, threads);
181
182 }
```

The image shows the code for the main function in the main file of the program. Firstly, the program asks the user for input for the size of the matrix and stores it. Then it does the same for the number of threads. After saving each it checks if the number of valid and sanitizes it if not to a standard 1000 or a 1. Then it creates the three matrices and calls the function to generate the two matrices discussed in Figure 8. Then it calls the functions discussed above and saves the output to the user created data structure as 4 different variables. Then it checks if the checksum across all multiplication sis the same and prints the appropriate message. Then three times it calls the functions to calculate the speedup and parallel portion for each experiment and prints the results.

Figure 7

Image of the code from the header file



```cpp
//File for all shared operations used on matrices and vectors but only declares them (no body)

#pragma once //Prevents multiple inclusions of this header file

#include <cstddef> //Importing the cstddef library for size_t
#include <string> //Importing the string library for std::string
#include <vector> //Importing the vector library for std::vector

using MatrixNa = std::vector<double>; //The Matrix used throughout the program

// Fills an n x n matrix with random values between 0 and 1 using the provided seed.
void MatrixFill(MatrixNa& mat, std::size_t size, unsigned int seed); //Unassigned means only positive larger numbers

//Multiplication function of AxB = C with single thread.
void MatrixMultiplySingle(const MatrixNa& A, const MatrixNa& B, MatrixNa& C, std::size_t size);

//Multiplication function of AxB = C with multiple threads.
void MatrixMultiplyMulti(const MatrixNa& A, const MatrixNa& B, MatrixNa& C, std::size_t size, std::size_t threads);

//Multiplication function of AxB = C with multiple threads and cache-friendly (reordered) loops.
void MatrixMultiplyMultiReordered(const MatrixNa& A, const MatrixNa& B, MatrixNa& C, std::size_t size, std::size_t threads);

//Multiplication function of AxB = C with multiple threads and cache blocking (the matrices are processed in small tiles).
void MatrixMultiplyMultiReorderedBlocked(const MatrixNa& A, const MatrixNa& B, MatrixNa& C, std::size_t size, std::size_t
threads);

//Speedup of a multi-threaded run compared to the single-thread run: T(single) / T(multi).
double CalculateSpeedup(double timeSingleThread, double timeMultiThread);

//Parallel fraction P of the program measured from the speedup (Amdahl's law solved for P).
double CalculateAmdahlParallelFraction(double speedup, std::size_t threads);
```

The image above shows the code from the header file. This file is used to functions declaration. At the beginning on line 3 there is a pre-processor directive which makes sure that the file is included only once in one compilation. Then all necessary imports are done for example the vector, string and a tool used for standardized sizing. Then the matrix variable is defined and all other functions which are explained above and below.

Figure 8

Image of the code for generating the matrices



```cpp
//File used to create matrices with random values

#include "mat_operations.h" //Including the header file for matrix operations
#include <iostream> //Including the input-output stream library for console input and output
#include <random> //Including the random library for generating random numbers

//Function to fill a matrix with random values
void MatrixFill(MatrixNa& mat, std::size_t size, unsigned int seed)
{
    mat.resize(size * size); //Sets the matrix size

    std::mt19937 generator(seed); //Using the Mersenne Twister random number generator with a seed for reproducibility
    std::uniform_real_distribution<double> distribution(0.0, 1.0); //Uniform distribution of random numbers between 0.0 and 1.0

    //Filling the matrix with random values
    for (std::size_t k = 0; k < size * size; ++k)
    {
        mat[k] = distribution(generator);      You, yesterday • Creating the first files, function for matrix c…
    }
}
```

The image above shows the code that is randomly generating the matrices. Firstly, as always the imports are made and here there is a new one called random which is the import for random library. Then there is

a function which firstly takes the size from the main function and multiples by itself to get the final matrix size. Then on line 12 the generator uses the Mersenne Twister which is a standard algorithm that produced random numbers. In here it is not truly random because a seed is sent though which will decide what numbers to use so every run will have the same matrices. The next line takes the generated numbers and transform them into a uniform, real numbers between 0 and 1. Then each number is placed in a loop into the matrix.

Figure 9

Image of the code which uses single thread to do multiplication of matrices

```
1  //Baseline that computes the multiplication on a single thread
2
3  #include "mat_operations.h" //Including the header file for matrix operations
4
5  //Function for single thread multiplication
6  void MatrixMultiplySingle(const MatrixNa& A, const MatrixNa& B, MatrixNa& C, std::size_t size)
7  {      You, yesterday • adding multiplication of matricies with single …
8      C.assign(size * size, 0.0); //Setting the size of the result matrix
9
10     for (std::size_t row = 0; row < size; ++row) //Each row of A
11     {
12         for (std::size_t col = 0; col < size; ++col) //Each column of B
13         {
14             double sum = 0.0;
15             for (std::size_t k = 0; k < size; ++k) //Walk along row of A and down column of B
16             {
17                 sum += A[row * size + k] * B[k * size + col];
18             }
19             C[row * size + col] = sum;
20         }
21     }
22 }
```

The image above shows the ode used to make a simple, single thread multiplication. The function takes the variables from the main function and then firstly assigned the matrix C with empty variables and resize the matrix to the correct size. Then the first loop iterates through the row from matrix A and the inner loop through each column from matrix B. Finally, the last loop walks through each value and sums it up and later saves to the matrix C.

Figure 10

Image of the code which uses multi thread to do multiplication of matrices

```cpp
//Computes the multiplication on multiple threads

#include "mat_operations.h" //Including the header file for matrix operations
#include <thread> //Including the thread library for multi-threading
#include <vector> //Including the vector library for std::vector


//Function for calculating rows [rowStart, rowEnd) of C, only used inside this file
static void MultiplyMatrixRows(const MatrixNa& A, const MatrixNa& B, MatrixNa& C, std::size_t size, std::size_t rowStart,
std::size_t rowEnd)
{
    for (std::size_t row = rowStart; row < rowEnd; ++row) //Each column of A
    {
        for (std::size_t col = 0; col < size; ++col) //Each column of B
        {
            double sum = 0.0;
            for (std::size_t k = 0; k < size; ++k) //Walk along row of A and down column of B
            {
                sum += A[row * size + k] * B[k * size + col];
            }
            C[row * size + col] = sum;
        }
    }
}
```

```cpp
//Function for multi thread multiplication
void MatrixMultiplyMulti(const MatrixNa& A, const MatrixNa& B, MatrixNa& C, std::size_t size, std::size_t threads)
{
    C.assign(size * size, 0.0); //Setting the size of the result matrix

    //Checking and sanitizing the number of threads input
    if (threads == 0)      You, yesterday • adding matrix multiplication with mutli threadi…
    {
        threads = 1;
    }
    else if (threads > size)
    {
        threads = size;
    }

    std::vector<std::thread> workerthreads; //Vector to hold the worker threads
    const std::size_t rowsperthread = size / threads; // rows every thread gets
    const std::size_t extra = size % threads; // if nr of rows dont divide equally the the extra rows needs to be assigned to
    some thread
    std::size_t start = 0;

    for (std::size_t t = 0; t < threads; ++t)
    {
        std::size_t end = start + rowsperthread;
        if (t < extra)
        {
            end = end + 1; // the first 'extra' threads go one row further
        }

        //Starting a new thread to compute the rows [start, end) of C
        workerthreads.emplace_back([&A, &B, &C, size, start, end]()
        {
            MultiplyMatrixRows(A, B, C, size, start, end);
        });

        start = end;
    }

    for (auto& thread : workerthreads)
    {
        thread.join(); //Wait for all threads to finish
    }
}
```

The image above shows the code for the multi-threaded multiplication matrix. The first function is the same as before with single thread but there is another function. The second function firstly checks if the

threads inputted and if it is zero it changes to a 1 and if the amount, of threads is higher than the size, it will be set equal tot eh size because there is no need to use more threads than the row/column of the matrix. Then a list is created also called vector in C++ to hold all the working threads inside. Then a calculation is done to calculate how many rows each thread will get and if there is any non-divisible number then the next line is calculating the left amount. Then a loop is created which assigned the ranges of rows/columns to each thread and also the extra columns/rows to each thread if needed. It keeps track of what is assigned and what is not by assigned the start to the end when it is finished with one thread. At the end the threads are joined together and outputted is suited together.

Figure 11

Image of the code which uses multi thread and reorder to do multiplication of matrices

```cpp
//Computes the multiplication on multiple threads with memory-optimised (reordered) loops

#include "mat_operations.h" //Including the header file for matrix operations
#include <thread> //Including the thread library for multi-threading
#include <vector> //Including the vector library for using std::vector


//Function for calculating rows [rowStart, rowEnd) of C with the loop order row, k, col
static void MultiplyMatrixRowsReordered(const MatrixNa& A, const MatrixNa& B, MatrixNa& C, std::size_t size, std::size_t rowStart, std::size_t rowEnd)
{
    for (std::size_t row = rowStart; row < rowEnd; ++row) //Every row in the whole A
    {
        for (std::size_t k = 0; k < size; ++k) //Each column in the whole A (or row in the whole B)
        {
            const double a = A[row * size + k]; //Loaded once and reused for the whole row of B
            for (std::size_t col = 0; col < size; ++col) //Walk along row k of B and along row of C
            {
                C[row * size + col] += a * B[k * size + col];
            }      You, yesterday • adding multithreading with memory optimizedf fu…
        }
    }
}
```

```cpp
//Function for multi thread multiplication with memory-optimised (reordered) loops
void MatrixMultiplyMultiReordered(const MatrixNa& A, const MatrixNa& B, MatrixNa& C, std::size_t size, std::size_t threads)
{
    C.assign(size * size, 0.0); //Setting the size of the result matrix and zeroing it (the += in the loop needs a zero start)

    //Checking and sanitizing the number of threads input
    if (threads == 0)
    {
        threads = 1;
    }
    else if (threads > size)
    {
        threads = size;
    }

    std::vector<std::thread> workerthreads; //Vector to hold the worker threads
    const std::size_t rowsperthread = size / threads; // rows every thread gets
    const std::size_t extra = size % threads; // if nr of rows dont divide equally the extra rows needs to be assigned to some thread
    std::size_t start = 0;

    for (std::size_t t = 0; t < threads; ++t)
    {
        std::size_t end = start + rowsperthread;
        if (t < extra)
        {
            end = end + 1; // the first 'extra' threads go one row further
        }

        workerthreads.emplace_back([&A, &B, &C, size, start, end]()
        {
            MultiplyMatrixRowsReordered(A, B, C, size, start, end);
        });

        start = end;
    }

    for (auto& thread : workerthreads)
    {
        thread.join(); // wait for all threads to finish
    }
}
```

The image above shows the code for the multi-threaded multiplication matrix with reordering. The second function is the same as before, but the first one is different. In the most part it is the same it's just one variable that changes which makes the transversal of row to row rather than column and row.

Figure 12

Image of the code which uses multi thread, reorder and slicing to do multiplication of matrices

```cpp
//Computes the multiplication on multiple threads with cache blocking (tiling)      You, 23 hours ago • adding the multi thread

#include "mat_operations.h" //Including the header file for matrix operations
#include <algorithm> //Including the algorithm library for std::min
#include <thread> //Including the thread library for multi-threading
#include <vector> //Including the vector library for std::vector

static const std::size_t blockSize = 64; //Size of one tile (blockSize x blockSize numbers), 64 x 64 doubles = 32 KB

//Function for calculating rows [rowStart, rowEnd) of C tile by tile, only used inside this file
static void MultiplyMatrixRowsBlocked(const MatrixNa& A, const MatrixNa& B, MatrixNa& C, std::size_t size, std::size_t
rowStart, std::size_t rowEnd)
{
    for (std::size_t rowBlock = rowStart; rowBlock < rowEnd; rowBlock += blockSize) //Selected block of rows of C to compute
    {
        const std::size_t rowLimit = std::min(rowBlock + blockSize, rowEnd); //Limit of the current block of rows to compute

        for (std::size_t kBlock = 0; kBlock < size; kBlock += blockSize) //Selected block of columns of A and rows of B to
        compute
        {
            const std::size_t kLimit = std::min(kBlock + blockSize, size); //Limit of the current block of columns of A and
            rows of B to compute

            for (std::size_t colBlock = 0; colBlock < size; colBlock += blockSize) //Selected block of columns of C to compute
            {
                const std::size_t colLimit = std::min(colBlock + blockSize, size); //Limit of the current block of columns of
                C to compute

                //Same loops as the reordered version, but only inside the current selected block
                for (std::size_t row = rowBlock; row < rowLimit; ++row)
                {
                    for (std::size_t k = kBlock; k < kLimit; ++k)
                    {
                        const double a = A[row * size + k]; // loaded once and reused along the tile row of B
                        for (std::size_t col = colBlock; col < colLimit; ++col)
                        {
                            C[row * size + col] += a * B[k * size + col];
                        }
                    }
                }
            }
        }
    }
}
```

```cpp
//Function for multi thread multiplication with memory-optimised (reordered) loops and cache blocking
void MatrixMultiplyMultiReorderedBlocked(const MatrixNa& A, const MatrixNa& B, MatrixNa& C, std::size_t size, std::size_t threads)
{
    C.assign(size * size, 0.0); //Setting the size of the result matrix and zeroing it (the += in the loop needs a zero start)

    //Checking and sanitizing the number of threads input
    if (threads == 0)
    {
        threads = 1;
    }
    else if (threads > size)
    {
        threads = size;
    }

    std::vector<std::thread> workerthreads; //Vector to hold the worker threads
    const std::size_t rowsperthread = size / threads; //Rows that every thread gets
    const std::size_t extra = size % threads; //If nr of rows dont divide equally the extra rows needs to be assigned to some thread
    std::size_t start = 0; //Starting row for the current thread

    //Creating the worker threads and assigning them their rows to compute
    for (std::size_t t = 0; t < threads; ++t)
    {
        std::size_t end = start + rowsperthread; //Ending row for the current thread
        if (t < extra)
        {
            end = end + 1; // the first 'extra' threads go one row further
        }

        //Creating a thread and assigning it the function to compute its rows
        workerthreads.emplace_back([&A, &B, &C, size, start, end]()
        {
            MultiplyMatrixRowsBlocked(A, B, C, size, start, end);
        });

        start = end;
    }

    //Waiting for all threads to finish their work
    for (auto& thread : workerthreads)
    {
        thread.join(); //Wait for all threads to finish and join together
    }
}
```

The image above shows the code for the multi-threaded multiplication matrix with reordering but also with blocking cache size. The second function is again the same as before, but the first function is different. The first function firstly does three loops which are the same loops as before just reordered. The first one decide which 64 block will be taken, then the second loop decides which row of A will be computed, and the last one choose which columns will be computed. In between there is also a constrain which does not allow the counter to go beyond the matrix size because there are case in which the size is not a multiple of 64. The inner two loops are the same as before, but only inside the current tile. It takes one value from A, multiplies it with a row segment of B, and adds the result into the row segment of C.

Figure 13

Image of the code used to calculate speedup and parallel portion

```cpp
//Calculations of speedup and Amdahl's law from measured times      You, 21 hours ago • adding laws calc

#include "mat_operations.h" //Including the header file for matrix operations

//Function for speedup of a multi-threaded run compared to the single-thread
double CalculateSpeedup(double secondsSingleThread, double secondsMultiThread)
{
    //Checking if the multi-threaded time is less than or equal to 0.0
    if (secondsMultiThread <= 0.0)
    {
        return 0.0; // protects against dividing by zero
    }
    return secondsSingleThread / secondsMultiThread;
}

//Function for parallel fraction P of the program measured from the speedup (Amdahl's law solved for P)
double CalculateAmdahlParallelFraction(double speedup, std::size_t threads)
{
    //Checking if the number of threads is less than 2 or if the speedup is less than or equal to 0.0
    if (threads < 2 || speedup <= 0.0)
    {
        return 0.0; // with one thread nothing can be measured
    }
    const double n = static_cast<double>(threads);
    return 100 * (1.0 - 1.0 / speedup) / (1.0 - 1.0 / n);
}

```

The image above shows the code for the functionality of calculating the speedup and parallel fraction. The first method checks if the time for multi thread is bigger than zero and if yest then it will return the single thread time divided by the multi thread which will give the speedup. Later in main it will be used to display the result. The other function checks if the number of threads is bigger than 1 and the speedup is bigger than 0 then it will return the parallel portion in percentages with reorder Amdahl's law.

12.5 C.2 – Program Execution Output

Figure 14

Image of the output from first cod execution

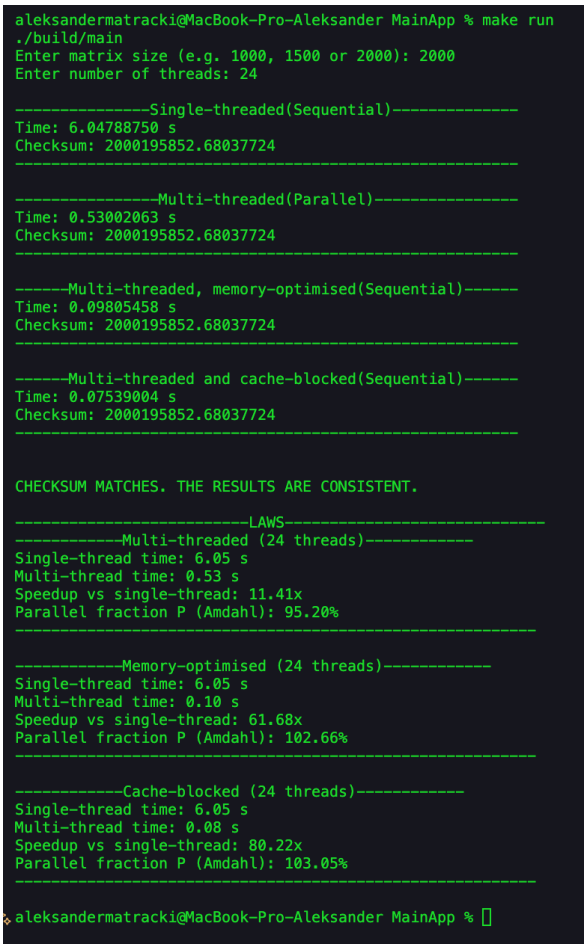


Figure 15

Image of the output from second cod execution

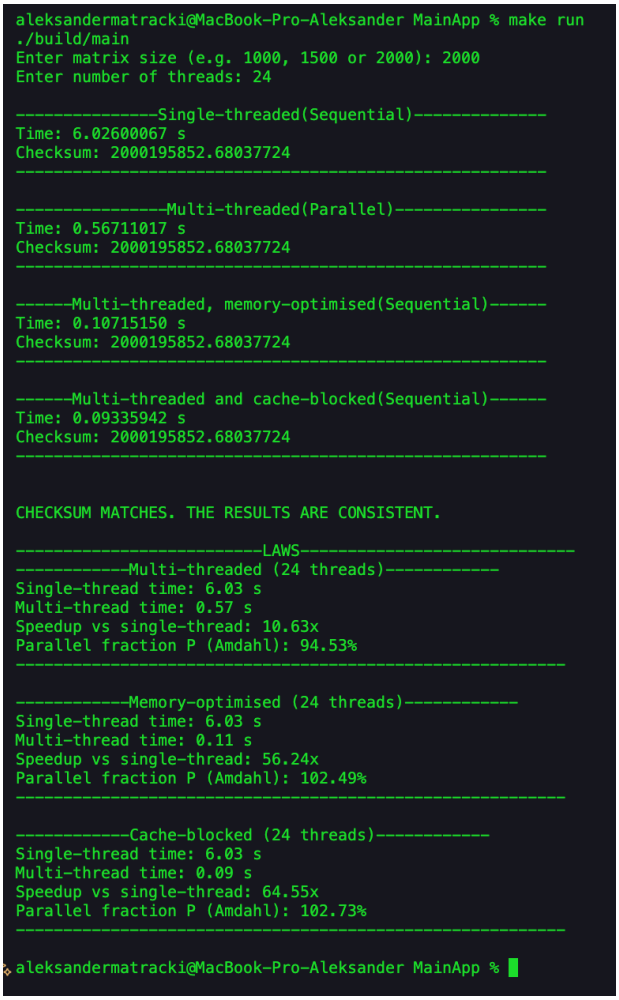


Figure 16

Image of the output from third cod execution

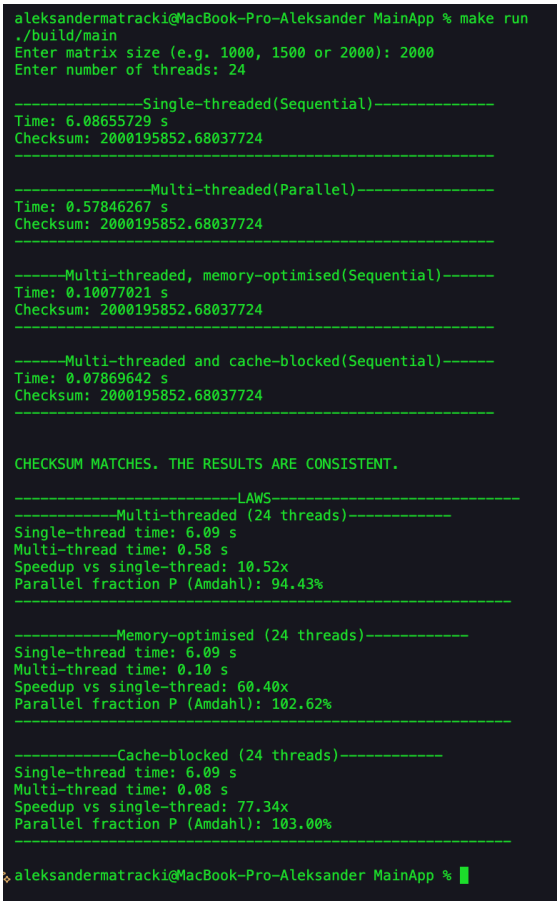


Figure 17

Image of the output from fourth cod execution

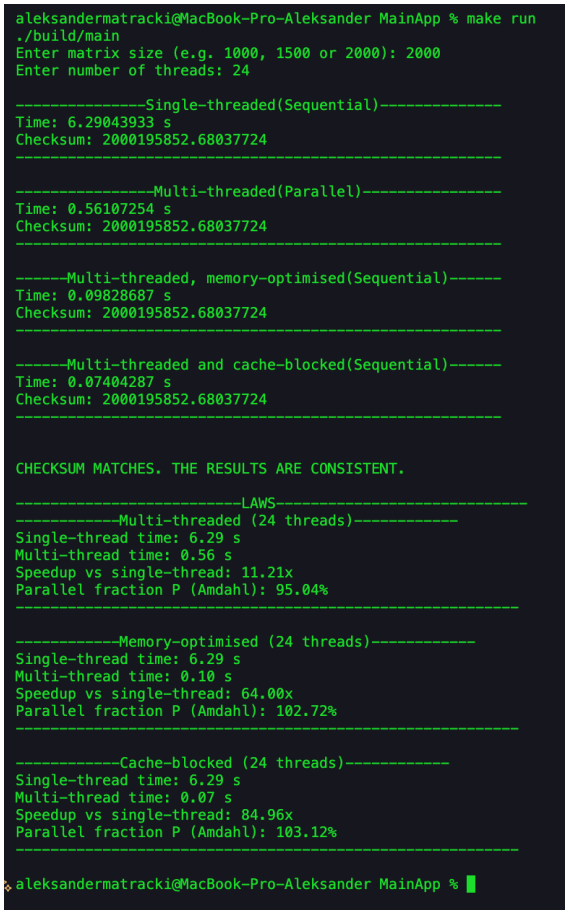


Figure 18

Image of the output from fifth cod execution

